

Discovery Search

Under the hood, Nia is an indexing and retrieval service with an MCP interface and an API. You point it at sources like GitHub repositories, framework or provider docs, SDK pages, PDF manuals, etc. We fetch and parse those with some simple heuristics for code structures, headings, and tables, then normalize them into chunks and build several indexes: a semantic index with embeddings for natural language queries; a symbol and usage index for functions, classes, types, and endpoints; a basic reference graph between files, symbols, and external docs; regex and file tree search for cases where you want deterministic matches over raw text.

When an agent calls Nia, it sends a natural language query plus optional hints like the current file path, stack trace, or repository. Nia runs a mix of BM25 style search, embedding similarity, and graph walks to rank relevant snippets, and can also return precise locations like “this function definition in this file and the three places it is used” instead of just a fuzzy paragraph. The calling agent then decides how to use those snippets in its own prompt. One Nia deployment can serve multiple agents and multiple projects at once. For example, you can have Cursor, Claude Code, and a browser based agent all pointed at the same Nia instance that knows about your monorepo, your internal wiki, and the provider docs you care about. We keep an agent agnostic session record that tracks which sources were used and which snippets the user accepted. Any MCP client can attach to that session id, fetch the current context, and extend it, so switching tools does not mean losing what has already been discovered.

—

provide detailed steps to build a Python project implementing the features described.

A practical way to build this is as a modular Python service with four main subsystems: ingestion, indexing, retrieval, and serving (HTTP + MCP). [1][2][3]

1. Plan project structure

Design a monorepo-friendly, service-oriented layout:

- `nia\_server/`: core package
 - `ingestion/`: Git, HTTP, filesystem, PDF, HTML parsers
 - `chunking/`: code/document chunking heuristics
 - `indexing/`: BM25, embeddings, symbol index, reference graph, regex/file-tree search
 - `retrieval/`: hybrid ranking, graph walks, snippet assembly
 - `session/`: session state and snippet-acceptance tracking
 - `mcp/`: MCP server implementation and schemas
 - `api/`: HTTP/JSON API (FastAPI)

- ``models/``: Pydantic models for queries, results, config
- ``storage/``: DB and vector-store abstraction
- ``scripts/``: CLI tools for crawling and backfilling sources
- ``tests/``: unit/integration tests

Use Python 3.11+, FastAPI for HTTP, SQLAlchemy + Postgres (or SQLite to start), and a vector DB (Qdrant, pgvector, or similar) for embeddings. [2][3][4]

2. Implement ingestion pipelines

1. **Source registry and config**

- Create a ``Source`` table with fields: ``id``, ``type`` (``git``, ``docs_http``, ``pdf``, ``wiki``), ``url_or_path``, ``metadata`` JSON, ``default_branch``, ``enabled``.
- Add a small CLI (Typer) to register sources and trigger ingestion runs.

2. **Fetchers**

- Git: use ``gitpython`` to clone/update repos into a local cache directory keyed by ``source_id``.
- HTTP docs/SDK pages: use ``httpx`` or ``requests`` with rate limiting; respect robots.txt and auth where required.
- PDFs: use ``pypdf`` or ``pdfminer.six`` to extract text and basic layout.
- Local filesystem: walk directories and read files directly.

3. **Parsers and structure extraction**

- Code (Python, TS/JS, Go, etc.):
 - Use language-specific parsers where possible (e.g., ``ast`` for Python, ``tree-sitter`` bindings for multi-language) to identify functions, classes, methods, types, and docstrings. [5][6]
- Markdown/HTML docs:
 - Parse headings, sections, and tables with ``markdown-it-py`` or ``beautifulsoup4``.
- PDFs:
 - Treat each page or logical section as a document, splitting by headings or large font changes where detectable.

4. **Normalization model**

- Define a ``Document`` model (DB + Pydantic): ``id``, ``source_id``, ``uri`` (e.g., ``repo://path/to/file.py#L10``), ``language``, ``content``, ``structured_blocks`` (JSON), ``metadata``.
- Normalize all parsed content into this internal format before chunking.

3. Chunking and symbol/usage extraction

1. **Chunking strategy**

- Implement language-aware chunkers:
 - Code: chunk by function/method/class or small groups of adjacent definitions; include surrounding comments and relevant imports. [5][6]

- Docs: chunk by heading + paragraph, keeping chunk sizes within a token budget (e.g., 512–1024 tokens). [7][4]

- Represent chunks as `Chunk` records with: `id`, `document\_id`, `text`, `start\_offset`, `end\_offset`, `symbol\_ids` (optional), `metadata` (headings, path, language).

2. **Symbol and usage index**

- Define `Symbol` model: `id`, `source\_id`, `name`, `kind` (`function`, `class`, `type`, `endpoint`, etc.), `signature`, `uri` (file + range), `container` (module/class), `language`.

- During parsing, extract:

- Definitions (where symbol is introduced).

- References/usages (call sites, type annotations, imports).

- Store usage relations in a `SymbolUsage` or `SymbolEdge` table: `from\_symbol\_id`, `to\_symbol\_id`, `kind` (`call`, `inherits`, `imports`, `endpoint\_calls`, etc.). [5][6]

- Link each `Chunk` to associated symbols (definitions and notable usages).

3. **Basic reference graph**

- Build a graph over nodes of type `file`, `symbol`, and `external\_doc\_section`.

- Use edges: `defines`, `uses`, `imports`, `links\_to` (for hyperlinks between docs), `same\_topic` (if same heading or section). [8][3]

- Store adjacency lists in a relational table or in a graph-friendly store (but Postgres tables are enough initially).

4. Build multiple indexes

1. **BM25 / sparse index**

- Use an inverted-index library (e.g., `whoosh`, `rank-bm25`, or `txtai`) to build a BM25 index over chunk texts. [1][9][10]

- Index fields: `text` (chunk content), plus boosted fields for `symbol\_names`, `file\_path`, `headings`. [9]

2. **Semantic embedding index**

- Use a sentence transformer or hosted embedding model to embed chunk texts. [4]

- Store embeddings in a vector store (Qdrant, pgvector, etc.) with metadata: `chunk\_id`, `source\_id`, `path`, `language`, `symbol\_ids`. [2][3]

- Add metadata filters (e.g., by repo, language, path prefix) to support targeted queries. [2]

3. **Symbol index**

- For symbol-name and signature search, maintain a dedicated index (could be:

- a trigram index in Postgres,

- a small BM25 index over symbol names and docstrings, or

- both). [5][6]

4. **Regex and file-tree search**

- Implement deterministic search over raw file contents using:

- direct regex search on stored file blobs for small to medium repos, or
- ripgrep-like subprocess calls against a local checkout for larger monorepos.
- Expose a file-tree API: list sources, directories, and files, and allow retrieval of raw file content.

5. Implement retrieval and ranking

1. **Query model and hints**

- Create a `SearchQuery` schema with fields:
 - `query_text` (natural language)
 - `source_filters` (repos, docs sets)
 - `path_hint` (current file path)
 - `stack_trace` (optional, for future call-stack-aware ranking)
 - `symbol_hint` (symbol name or language)
 - `mode` (`'auto'`, `'symbol'`, `'regex'`, etc.).

2. **Hybrid retrieval pipeline**

- Step 1: BM25 retrieval of top N chunks. [1][9][3]
- Step 2: Semantic retrieval of top M chunks using embeddings. [2][3][4]
- Step 3: Symbol search if query looks like a symbol (e.g., contains `'Class.method'`, `'func('`, or path-like patterns). [5][6]
- Step 4: Merge candidates:
 - Normalize scores (e.g., z-score or min-max) across BM25 and embeddings. [2][3]
 - Compute a weighted sum, with possible boosts for:
 - chunks in the same repo or path subtree as `'path_hint'`.
 - symbols directly matching query tokens.
- Step 5: Use the reference graph to expand:
 - Add neighbors such as direct callees, callers, or related doc sections. [8][3]

3. **Precise location results**

- For each candidate chunk, map back to:
 - file path, start/end line numbers;
 - associated symbols and their definitions/usages.
- Design response flavors:
 - “Snippet list” (small text chunks plus metadata).
 - “Symbol results” (definition location plus a small set of usage locations, e.g., top 3).

6. Session and context tracking

1. **Session store**

- Implement a `Session` model: `'id'`, `'created_at'`, `'user_id'` (optional), `'active'`.
- Implement `SessionEvent` or `SessionSnippet` records capturing:
 - `'session_id'`, `'query'`, `'result_ids'`, `'accepted_snippet_ids'`, timestamps, and source metadata.

2. ****Agent-agnostic session IDs****

- Enable agents to pass a `session\_id` to both HTTP and MCP APIs.
- Provide endpoints to:
 - fetch past queries and accepted snippets for a session,
 - append new events,
 - use session context as soft signals in ranking (e.g., prefer previously used sources or paths).

3. ****Context reconstruction****

- Add an API call like `GET /sessions/{id}/context` that returns a compact structure summarizing the session: selected snippets, active sources, and key symbols, so any agent can reconstruct context easily.

7. HTTP API design

Use FastAPI and Pydantic:

- `POST /search`
 - Input: `SearchQuery` + optional `session\_id`.
 - Output: ranked snippets, symbol definitions/usages, graph neighbors.
- `POST /symbols/search`
 - Input: symbol string and filters.
 - Output: symbol definitions and usage sites.
- `GET /sources`, `POST /sources`, `POST /sources/{id}/ingest`
 - Manage and trigger ingestion.
- `GET /files/tree`, `GET /files/content`
 - Navigate and fetch raw file contents.
- `GET /sessions/{id}`, `GET /sessions/{id}/context`, `POST /sessions/{id}/events`
 - Session interactions.

Use dependency-injected services for `Retriever`, `Indexer`, `SessionStore`, and `SourceManager` to keep the API layer thin.

8. MCP server implementation

1. ****Understand MCP roles****

- MCP defines a host–client–server architecture where the MCP server exposes tools and resources via JSON-schema–described capabilities, and clients connect these to LLM hosts.
- [11][12][13][14]

2. ****Choose MCP server library****

- Use an existing MCP server helper library if available (e.g., from the official MCP org or LangChain's MCP integration) to manage handshakes, capability discovery, and request/response plumbing. [11][15][13][14]

3. **Define MCP tools**

- Define tools like:
 - ``search_docs``: map to ``/search`` in semantic mode.
 - ``search_symbols``: map to symbol-search API.
 - ``get_file``: retrieve file content by ``uri``.
 - ``get_session_context``: expose session summaries.
- Provide JSON Schemas for input/output so that MCP hosts (e.g., Cursor, Claude Code, browser-based agents) can call them. [11][15][13][14]

4. **Implement server process**

- Run the MCP server as a separate process or sidecar that connects to the same DB and vector store as the HTTP API. [11][12]
- Implement handlers that:
 - parse MCP tool invocations,
 - call internal services (``Retriever``, ``SessionStore``, ``SourceManager``),
 - return structured responses tailored for LLM consumption.

9. Multi-project and multi-agent support

1. **Multi-project isolation and filtering**

- Tag all sources, documents, chunks, and symbols with ``project_id`` or ``tenant_id``.
- Ensure all retrieval queries must specify or infer a project scope, so a single deployment can serve multiple repos and doc sets safely.

2. **Agent-agnostic behavior**

- Avoid agent-specific logic; treat all clients as generic.
- Session IDs and project IDs come from clients (Cursor, Claude Code, browser agent), which can all attach to the same Nia instance.

3. **Config and deployment**

- Provide a single configuration file (e.g., YAML) listing DB connection, vector store, embedding model, and ingestion schedules.
- Containerize the service with Docker; run background workers (Celery or RQ) for ingestion and indexing jobs, and a web server for HTTP + MCP.

10. Testing, observability, and iteration

1. **Testing**

- Unit tests for parsers, chunkers, symbol extraction, and ranker.

- Integration tests using a small sample repo and docs, verifying that queries return the expected definitions and usages.

2. **Observability**

- Add structured logging for ingestion runs, index updates, and search requests.
- Track metrics: ingestion latency, index size, search latency, and result click/accept rates (via `SessionEvent`).

3. **Iterative refinement**

- Start with BM25 + embeddings hybrid retrieval, then gradually incorporate more sophisticated graph walks and heuristic boosts as usage data accumulates. [8][2][3][4]

These steps give you a concrete path from empty repository to a working Python service that ingests multiple sources, builds hybrid indexes plus symbol and reference graphs, exposes precise code/doc locations, and serves multiple MCP-enabled agents over a shared, session-aware context.

Citations:

[1] What is BM25 (Best Matching 25) Algorithm

<https://www.geeksforgeeks.org/nlp/what-is-bm25-best-matching-25-algorithm/>

[2] Metadata Filtering and Hybrid Search for Vector Databases

<https://www.dataquest.io/blog/metadata-filtering-and-hybrid-search-for-vector-databases/>

[3] generative-ai/embeddings/hybrid-search.ipynb at main

<https://github.com/GoogleCloudPlatform/generative-ai/blob/main/embeddings/hybrid-search.ipynb>

[4] Semantic Search — Sentence Transformers documentation

https://sbert.net/examples/sentence_transformer/applications/semantic-search/README.html

[5] A hands-on introduction to static code analysis

<https://deepsources.com/blog/introduction-static-code-analysis>

[6] pyan is a Python module that performs static analysis ... <https://github.com/davidfraser/pyan>

[7] BM25(llamaindex) https://aiengineering.academy/RAG/01_BM25_RAG/notebook/

[8] Classic Topic Modeling with BM25

<https://dev.to/neuml/classic-topic-modeling-with-bm25-33ep>

[9] Enhance Your LLM Agents with BM25: Lightweight ...

<https://towardsai.net/p/artificial-intelligence/enhance-your-llm-agents-with-bm25-lightweight-retrieval-that-works>

[10] How to Create a BM25 Index in Python with Rank BM25 (Search Engine)

<https://www.youtube.com/watch?v=ysvpxiPAHLg>

[11] The Model Context Protocol (MCP): Deep Dive

<https://www.analytical-software.de/en/the-model-context-protocol-mcp-deep-dive-into-structure-and-concepts/>

[12] What is the Model Context Protocol (MCP)? - Model Context ...

<https://modelcontextprotocol.io>

[13] Build an MCP server <https://modelcontextprotocol.io/docs/develop/build-server>

- [14] Model Context Protocol <https://github.com/modelcontextprotocol>
- [15] Model Context Protocol (MCP) - Docs by LangChain
<https://docs.langchain.com/oss/python/langchain/mcp>
- [16] BM25 Retriever | LlamaIndex Python Documentation
https://developers.llamaindex.ai/python/examples/retrievers/bm25_retriever/
- [17] How to Inspect Function and Class Signatures in Python?
https://www.gaohongnan.com/playbook/how_to_inspect_function_and_class_signatures.html
- [18] How to perform static program analysis in Python for ...
<https://stackoverflow.com/questions/56175087/how-to-perform-static-program-analysis-in-python-for-finding-functions-with-cert>
- [19] We created an open-source semantic search Python ...
https://www.reddit.com/r/Python/comments/15d1aa5/we_created_an_opensource_semantic_search_python/
- [20] Python static code analysis <https://rules.sonarsource.com/python>

—

Design the high level architecture and components for Nia in Python

Nia in Python is best structured as a modular service with four layers: ingestion & normalization, indexing, retrieval & ranking, and serving (HTTP + MCP), all backed by shared storage and a session store.[1][2][3]

Core architecture overview

At a high level, Nia consists of:

- An ingestion layer that connects to sources (Git, docs, PDFs, wikis), parses them, and normalizes them into internal documents and chunks.
- An indexing layer that builds and maintains multiple indexes: BM25/sparse, dense embeddings, symbol/usage index, reference graph, and regex/file-tree search structures.[1][2][4]
- A retrieval layer that runs hybrid search (BM25 + embeddings + graph signals) and assembles precise locations (file, symbol, usage).
- A serving layer exposing a Python HTTP API (e.g. FastAPI) and a separate MCP server process, both talking to the same storage and indices.[5][6][7]

Ingestion and normalization layer

Components:

- ****Source Manager****: manages configured sources (repos, doc sets, PDFs) and schedules ingestion jobs.
- ****Fetchers****: Git fetcher (local clones), HTTP fetcher (docs/SDK pages), filesystem/PDF fetchers.

- **Parsers**: language-aware parsers for code, and HTML/Markdown/PDF parsers for docs, extracting headings, sections, tables, and basic code structures.[1][8]
- **Normalizer**: converts all parsed content into a common `Document` model (URI, content, language, structured blocks, metadata), ready for chunking and indexing.

Data flow: Source → Fetcher → Parser → Normalizer → `Document` table or collection in the primary database (e.g. Postgres) for further processing.[9][10]

Indexing and storage layer

Components:

- **Chunker**: splits documents into semantically meaningful chunks (functions/classes for code, heading+paragraph for docs) with references back to original files and line ranges.[1][8]
- **Symbol & Usage Extractor**: builds a symbol catalog for functions, classes, types, and endpoints plus their usage edges (defines, calls, imports, inherits).
- **Hybrid Search Indexes**:
 - BM25/sparse index over chunk text and symbol names (e.g. in an inverted-index engine).
 - Dense embedding index over chunk text stored in a vector database (e.g. pgvector/Qdrant) with metadata filters (repo, language, path).[2][3][4]
- **Reference Graph Store**: graph over files, symbols, and doc sections (nodes) with edges like defines, uses, imports, links_to.
- **Regex/File-tree Store**: efficient storage for raw file contents and directory structure, plus support for regex and path-based search.

All of these share a **metadata database** (SQL) that tracks sources, documents, chunks, symbols, and graph edges, while BM25 and vector indices store references to chunk IDs and metadata.[9][3][11]

Retrieval, ranking, and session layer

Components:

- **Query Router**: inspects the incoming natural language query plus hints (path, stack trace, repo) and chooses strategies (lexical, semantic, symbol, regex, or combinations).
- **Hybrid Ranker**: runs BM25 and dense search in parallel, then merges results using a fusion method (e.g. weighted scores or reciprocal rank fusion) and applies boosts based on hints and session context.[3][4][11]
- **Graph Expander**: takes top chunks/symbols and walks the reference graph to add related definitions, callers, callees, and linked docs.
- **Location Resolver**: maps chunks and symbols to concrete locations (file path, range) plus usage sites.
- **Session Store**: records sessions keyed by `session\_id`, storing queries, returned snippets, and user-accepted snippets so any agent can reconstruct or extend context later.[5][6][7]

Data flow: Query + hints → Query Router → BM25 & vector search → Hybrid Ranker → Graph Expander → Location Resolver → Result set, with Session Store logging selections and providing soft signals for subsequent ranking.[3][4]

Serving layer: HTTP API and MCP server

Components:

- **HTTP API (FastAPI)**:
 - `POST /search`: main hybrid search entry point returning ranked snippets and symbol locations.
 - `POST /symbols/search`, `GET /files/tree`, `GET /files/content`, `GET/POST /sessions` to expose symbol index, file tree, and session context to tools and agents.
- **MCP Server Process**: a separate Python process implementing MCP server semantics, exposing Nia's capabilities as MCP tools like `search_docs`, `search_symbols`, `get_file`, and `get_session_context` over JSON-RPC.[5][6][12][13]
- **Multi-tenant/Project Isolation**: project or tenant identifiers carried through all layers (sources, documents, chunks, indices, queries) so a single Nia deployment can serve multiple repos and agents safely.

The MCP server and HTTP API are thin facades over shared application services (Retriever, Indexer, Source Manager, Session Store), following the MCP guidance of running servers as isolated processes that expose focused capabilities to multiple clients and agents.[5][6][7][12]

Citations:

- [1] How to Build a Semantic Search Engine in Python
<https://www.deepset.ai/blog/how-to-build-a-semantic-search-engine-in-python>
- [2] How to Implement Semantic Search in Python Step by Step - TiDB
<https://www.pingcap.com/article/semantic-search-python-step-by-step/>
- [3] Hybrid search using vectors and full text in Azure AI Search
<https://learn.microsoft.com/en-us/azure/search/hybrid-search-overview>
- [4] Hybrid Search Explained <https://weaviate.io/blog/hybrid-search-explained>
- [5] Architecture - Model Context Protocol
<https://modelcontextprotocol.io/specification/2025-03-26/architecture>
- [6] Architecture - Model Context Protocol
<https://modelcontextprotocol.io/specification/2025-06-18/architecture>
- [7] The Architectural Elegance of Model Context Protocol (MCP)
<https://thelarchitect.com/blog/the-architectural-elegance-of-model-context-protocol-mcp/>
- [8] Building a Semantic Search Engine for Internal Documentation
<https://blog.devgenius.io/building-a-semantic-search-engine-for-internal-documentation-a-comprehensive-guide-270efa9a30a0>
- [9] What Is Semantic Search With Filters and How to ...
<https://www.tigerdata.com/blog/what-is-semantic-search-with-filters-and-how-to-implement-it-with-pgvector-and-python>

- [10] Build a semantic search engine in Python - Vikas Paruchuri
<https://www.vikas.sh/post/semantic-search-guide>
- [11] Building effective hybrid search in OpenSearch: Techniques and ...
<https://opensearch.org/blog/building-effective-hybrid-search-in-opensearch-techniques-and-best-practices/>
- [12] Introducing the Model Context Protocol - Anthropic
<https://www.anthropic.com/news/model-context-protocol>
- [13] What Is the Model Context Protocol (MCP) and How It Works
<https://www.descope.com/learn/post/mcp>
- [14] How to Build Your First Semantic Search System: My Step- ...
<https://mlops.community/how-to-build-your-first-semantic-search-system-my-step-by-step-guide-with-code/>
- [15] Introduction to Semantic Search with Python and OpenAI API
<https://dev.to/carolinamonte/introduction-to-semantic-search-with-python-and-openai-api-efg>
- [16] Tutorial: Hybrid search with BM25 in KDB-X AI libraries - Kx Systems
<https://kx.com/blog/tutorial-hybrid-search-with-bm25-in-kdb-x-ai-libraries/>
- [17] sagar8080/semantic-search-system <https://github.com/sagar8080/semantic-search-system>
- [18] Building an Index that supports a hybrid search comprising of Full ...
https://github.com/run-llama/llama_index/discussions/9837
- [19] Building a Semantic Code History Search with LanceDB
<https://blog.continue.dev/building-a-semantic-code-history-search-with-lancedb/>
- [20] Semantic Search — Sentence Transformers documentation
https://sbert.net/examples/sentence_transformer/applications/semantic-search/README.html

—

Design a scalable ingestion and preprocessing pipeline for Nia

A scalable ingestion and preprocessing pipeline for Nia should be a multi-stage, message-driven system that separates source discovery, fetch, parse, normalize, chunk, and index update, with each stage horizontally scalable and failure-isolated.[1][2][3]

High-level pipeline architecture

The pipeline can be modeled as an event-driven DAG:

- **Control plane**: manages source configs, schedules, and workflow orchestration.
- **Data plane**: workers subscribed to queues for each stage (fetch → parse → normalize → chunk → embed/index).

- A central metadata store (SQL) tracks document state, and separate stores handle raw blobs and vector/BM25 indices.[1][4][5]

Using queues (e.g., Redis, RabbitMQ, Kafka) allows each stage to scale independently and recover from failures while preserving ordering guarantees per document or source where needed.[1][4]

Stages and components

1. **Source discovery and scheduling**

- A `SourceRegistry` in Postgres stores repos, HTTP doc roots, file shares, and PDFs, with configuration like poll interval, last seen revision, and parsing options.[6][7]

- A `Scheduler` periodically enqueues “ingestion jobs” per source with a version token (e.g., Git commit hash, HTTP ETag, or directory snapshot ID).

2. **Fetchers (I/O heavy workers)**

- Workers consume `FetchJob` messages and:

- For Git: fetch and diff against the last indexed commit to identify changed/removed files.

- For HTTP/SDK docs: crawl sitemaps or link lists, respecting robots and using

ETags/Last-Modified to detect changes.[6][8]

- For PDFs and file shares: list and hash files to detect new/updated ones.

- Results are written as `RawDocument` entries (URI, source_id, blob location, content-type, checksum) and “parse” tasks are enqueued per file.[4][8]

3. **Parsers (CPU-heavy workers)**

- A `ParserRouter` looks at MIME type and file extension and dispatches to specialized parsers:

- Code parsers (Python/TS/etc.) using language-specific tools to extract AST, symbols, docstrings.

- Markdown/HTML parsers capturing headings, sections, links, and tables.[6][8]

- PDF parsers or Docling-like tools to get structured JSON with text blocks and tables.[6][9]

- Each parser outputs a structured `ParsedDocument` (blocks, headings, symbol candidates, metadata) stored in the DB or an object store, and emits a `NormalizeJob` message.

4. **Normalization and de-duplication**

- A `Normalizer` worker converts `ParsedDocument` to the internal `Document` model (stable URI, language, logical sections) and ensures consistent text cleaning (case, whitespace, normalization) that will be shared across BM25 and embedding flows.[10][11]

- It performs deduplication (e.g., by content hash) to avoid re-chunking identical docs and can reuse embeddings when content is unchanged, which is critical for cost and throughput at scale.[12][10]

5. **Chunking and preprocessing**

- A `Chunker` applies configurable strategies per document type:

- Code: split by function/class, grouping neighboring small definitions and associated comments.
- Docs: heading-based chunks with size limits (token or character based), preserving structural markers.[13][7]
- For each chunk, it emits `Chunk` records (text, offsets, metadata) and enqueues `IndexJob` messages that cover both lexical and embedding indexing.[14][13]

6. **Indexing workers (BM25 + embeddings)**

- **Lexical/BM25 indexer**: tokenizes chunk text, applies stopword filters, and updates an inverted index (e.g., local engine or external search service), using the normalized text so lexical and vector views stay consistent.[14][10][15]
- **Embedding indexer**: batches chunks and calls the embedding model (local or remote), with caching keyed by content hash to avoid recomputation, then upserts vectors into the vector DB (Qdrant, pgvector, Milvus, etc.).[13][12][3]
- Both indexers must be idempotent and able to handle out-of-order messages by checking the latest document version in metadata before applying changes.[4][16]

Orchestration, scaling, and reliability

- **Workflow orchestration**

- Use a workflow engine (e.g., Celery beat + queues, DBOS-style workflows, or Ray DAGs) to define ingestion as a resilient multi-step process: fetch → parse → normalize → chunk → index, with retries and checkpointing per document.[2][4][3]

- **Horizontal scaling**

- Run separate worker pools for `fetch`, `parse`, `chunk`, and `index` queues so each can scale based on its CPU/IO profile; for large indexing jobs, auto-scale embedding workers and use batch inference.[13][2][3]

- **Backpressure and prioritization**

- Limit concurrent jobs per source to avoid overloading Git servers or doc sites.
- Introduce priority queues (e.g., “hot” repos vs archival sources) so frequently used codebases index faster.

- **State tracking and idempotency**

- Each document and chunk row includes a `version` or `generation` number; workers only apply updates if the job's version matches or exceeds the stored one, avoiding stale overwrites.[4][16]
- Deletions and renames emit tombstone events so indexers can remove or update entries.

Python implementation outline

- Use a central Postgres (or similar) DB for metadata, plus:
 - object storage (S3-like) for raw and parsed blobs,
 - a vector DB for embeddings,

- an optional external lexical engine (OpenSearch/Elasticsearch) or an internal BM25 implementation.[1][12][17]
- Implement workers with Celery, RQ, or Ray for distributed execution, with tasks corresponding to the stages above.
- Provide an “ingestion API” and CLI that:
 - registers sources, triggers full or incremental runs, and reports progress via document counts and per-stage metrics.[1][4][5]

This architecture gives Nia a robust ingestion path that can grow from a single-node Python process to a distributed pipeline capable of handling large monorepos, extensive internal wikis, and big PDF/manual corpora while keeping BM25, embeddings, and structural metadata in sync.

Citations:

- [1] Building a Scalable Document Processing Pipeline With ... - MongoDB
<https://www.mongodb.com/company/blog/technical/building-scalable-document-processing-pipeline-llamaparse-confluent-cloud>
- [2] Building Scalable RAG Pipelines with Ray and Anyscale
<https://www.anyscale.com/blog/rag-pipelines-how-to>
- [3] Scaling Semantic Search Pipelines with Apache Spark and Vector ...
<https://pub.aimind.so/scaling-semantic-search-pipelines-with-apache-spark-and-vector-databases-24e5724af2da>
- [4] Document Ingestion Pipeline | DBOS Docs
<https://docs.dbos.dev/python/examples/document-detective>
- [5] Welcome to LlamaIndex ! | LlamaIndex Python Documentation
<https://developers.llamaindex.ai/python/framework/>
- [6] Building Document Parsing Pipelines with Python - SA Space
<https://saspace.substack.com/p/building-document-parsing-pipelines>
- [7] Data ingestion - .NET - Microsoft Learn
<https://learn.microsoft.com/en-us/dotnet/ai/conceptual/data-ingestion>
- [8] Tutorial: Preprocessing Different File Types - Haystack
https://haystack.deepset.ai/tutorials/30_file_type_preprocessing_index_pipeline
- [9] Docling - Docs by LangChain
https://docs.langchain.com/oss/python/integrations/document_loaders/docling
- [10] How do I implement BM25 alongside vector search? - Milvusmilvus.io › ai-quick-reference › how-do-i-implement-bm25-alongside-vect...
<https://milvus.io/ai-quick-reference/how-do-i-implement-bm25-alongside-vector-search>
- [11] Comparing Lexical and Semantic Vector Search Methods ...
<https://arxiv.org/html/2505.11582v2>
- [12] Building a Semantic Search System with Apache SeaTunnel and ...
<https://dev.to/seatunnel/building-a-semantic-search-system-with-apache-seatunnel-and-amazon-bedrock-4dof>
- [13] How To Build High-Performance RAG Pipelines That Scale
<https://www.deepchecks.com/build-high-performance-rag-pipelines-scale/>

[14] BM25(IIamaindex) - AI Engineering Academy
https://aiengineering.academy/RAG/01_BM25_RAG/notebook/

[15] Mastering BM25: A Deep Dive into the Algorithm and Its ...
<https://zilliz.com/learn/mastering-bm25-a-deep-dive-into-the-algorithm-and-application-in-milvus>

[16] The Architect's Guide to Production RAG: Navigating Challenges ...
<https://www.ragie.ai/blog/the-architects-guide-to-production-rag-navigating-challenges-and-building-scalable-ai>

[17] Scalable RAG PDF: From AWS S3 Buckets to Data Cloud Ingestion ...
<https://www.linkedin.com/pulse/scalable-rag-pdf-from-aws-s3-buckets-data-cloud-ingestion-werner-yyjue>

[18] RAG Pipeline: Example, Tools & How to Build It - lakeFS
<https://lakefs.io/blog/what-is-rag-pipeline/>

[19] Reranking in Hybrid Search
<https://qdrant.tech/documentation/advanced-tutorials/reranking-hybrid-search/>

[20] Elser
<https://www.elastic.co/docs/solutions/search/semantic-search/semantic-search-elser-ingest-pipeline>

—

Implement the basic features for Nia in Python and provide the complete code.

Below is a minimal but working “Nia v0” in a single file, showing basic ingestion, chunking, BM25 + semantic search, and a FastAPI HTTP API over it.[1][2][3]

Features included

- Ingest plain text “documents” via an API.
- Simple paragraph-based chunking.
- BM25 lexical index using `rank-bm25`. [4][5]
- Semantic index using `sentence-transformers` + cosine similarity (in-memory). [1][2]
- Hybrid retrieval: BM25 + embeddings with simple score fusion.
- FastAPI endpoints: `POST /documents`, `GET /search`. [6][3][7]

Complete code (single file: `nia\_basic.py`)

```
```python
"""
```

Minimal Nia prototype: ingestion + chunking + BM25 + semantic search + HTTP API.

Requirements (install with pip):

```
pip install fastapi uvicorn[standard] rank-bm25 sentence-transformers scikit-learn
```

Run:

```
uvicorn nia_basic:app --reload
```

Then:

1) Add documents:

```
curl -X POST "http://localhost:8000/documents" \
-H "Content-Type: application/json" \
-d '{"title": "Example", "content": "First paragraph.\n\nSecond paragraph about Python
and FastAPI."}'
```

2) Search:

```
curl "http://localhost:8000/search?q=python"
""
```

```
from typing import List, Dict, Optional
from dataclasses import dataclass, field
import uuid
```

```
from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
```

```

Data models (in-memory store)

```

```
@dataclass
class Chunk:
 id: str
 document_id: str
 text: str
 position: int # position of chunk inside document
 metadata: Dict[str, str] = field(default_factory=dict)
```

```
@dataclass
class Document:
 id: str
 title: str
 content: str
 chunks: List[Chunk] = field(default_factory=list)
```

```
class DocumentCreate(BaseModel):
 title: str
 content: str
```

```
class SearchResult(BaseModel):
 chunk_id: str
 document_id: str
 title: str
 text: str
 score: float
```

```
class SearchResponse(BaseModel):
 query: str
 results: List[SearchResult]
```

```

Simple in-memory Nia core engine

```

```
class NiaCore:
 """
 Minimal in-memory Nia core.
 - Stores documents and chunks.
 - Maintains BM25 and embedding indices.
 - Supports hybrid search over chunks.
 """

 def __init__(self, use_embeddings: bool = True):
 self.documents: Dict[str, Document] = {}
 self.chunks: Dict[str, Chunk] = {}

 # BM25 index inputs
```

```

self._bm25_corpus: List[List[str]] = [] # tokenized chunk texts
self._bm25_chunks: List[str] = [] # chunk_ids aligned with corpus
self._bm25: Optional[BM25Okapi] = None

Embedding index inputs
self.use_embeddings = use_embeddings
self._embed_model: Optional[SentenceTransformer] = None
self._embed_matrix = None # shape: (num_chunks, dim)
self._embed_chunk_ids: List[str] = []

if self.use_embeddings:
 # Tiny, fast model suitable for prototypes. [web:66]
 self._embed_model = SentenceTransformer("all-MiniLM-L6-v2")

Ingestion

def add_document(self, title: str, content: str) -> str:
 """
 Add a document and update indices.
 Splits content into paragraph-like chunks (separated by blank lines).
 """
 doc_id = str(uuid.uuid4())
 doc = Document(id=doc_id, title=title, content=content)
 self.documents[doc_id] = doc

 # Simple chunking: split on blank lines.
 raw_chunks = [p.strip() for p in content.split("\n\n") if p.strip()]
 for idx, chunk_text in enumerate(raw_chunks):
 chunk_id = str(uuid.uuid4())
 chunk = Chunk(
 id=chunk_id,
 document_id=doc_id,
 text=chunk_text,
 position=idx,
 metadata={"title": title},
)
 doc.chunks.append(chunk)
 self.chunks[chunk_id] = chunk

 # Rebuild indices naively (fine for small prototype).
 self._rebuild_bm25_index()
 if self.use_embeddings:

```

```

 self._rebuild_embedding_index()

 return doc_id

Indexing

@staticmethod
def _tokenize(text: str) -> List[str]:
 """Very simple whitespace tokenizer; extend for production."""
 return text.lower().split()

def _rebuild_bm25_index(self) -> None:
 self._bm25_corpus = []
 self._bm25_chunks = []

 for chunk_id, chunk in self.chunks.items():
 tokens = self._tokenize(chunk.text)
 if not tokens:
 continue
 self._bm25_corpus.append(tokens)
 self._bm25_chunks.append(chunk_id)

 if self._bm25_corpus:
 # Build BM25 index over chunk texts. [web:71][web:74]
 self._bm25 = BM25Okapi(self._bm25_corpus)
 else:
 self._bm25 = None

def _rebuild_embedding_index(self) -> None:
 if not self.use_embeddings or self._embed_model is None:
 return

 texts = []
 chunk_ids = []
 for chunk_id, chunk in self.chunks.items():
 text = chunk.text.strip()
 if not text:
 continue
 texts.append(text)
 chunk_ids.append(chunk_id)

 if not texts:

```

```

 self._embed_matrix = None
 self._embed_chunk_ids = []
 return

 # Compute embeddings for all chunks. [web:29][web:66]
 embeddings = self._embed_model.encode(texts, show_progress_bar=False)
 self._embed_matrix = embeddings
 self._embed_chunk_ids = chunk_ids

Search

def search(
 self,
 query: str,
 k: int = 5,
 alpha: float = 0.6,
) -> List[SearchResult]:
 """
 Hybrid search:
 - BM25 over chunks.
 - Embedding similarity over chunks.
 - Score fusion: $\alpha * \text{bm25_norm} + (1 - \alpha) * \text{embed_norm}$.
 """

 if not self.chunks:
 return []

 query_tokens = self._tokenize(query)

 # BM25 scores
 bm25_scores: Dict[str, float] = {}
 if self._bm25 is not None and query_tokens:
 scores = self._bm25.get_scores(query_tokens) # aligned with _bm25_chunks
 for chunk_id, score in zip(self._bm25_chunks, scores):
 bm25_scores[chunk_id] = float(score)

 # Embedding scores
 embed_scores: Dict[str, float] = {}
 if self.use_embeddings and self._embed_model is not None and self._embed_matrix is not
None:
 q_emb = self._embed_model.encode([query], show_progress_bar=False)
 sims = cosine_similarity(q_emb, self._embed_matrix)[0] # shape: (num_chunks,)

```

```

 for chunk_id, score in zip(self._embed_chunk_ids, sims):
 embed_scores[chunk_id] = float(score)

Normalize scores to [0,1] per signal (min-max).
def normalize(d: Dict[str, float]) -> Dict[str, float]:
 if not d:
 return {}
 vals = list(d.values())
 lo, hi = min(vals), max(vals)
 if hi == lo:
 return {k: 1.0 for k in d}
 return {k: (v - lo) / (hi - lo) for k, v in d.items()}

bm25_norm = normalize(bm25_scores)
embed_norm = normalize(embed_scores)

Combine scores
combined: Dict[str, float] = {}
candidate_ids = set(bm25_norm.keys()) | set(embed_norm.keys())
for cid in candidate_ids:
 s_bm25 = bm25_norm.get(cid, 0.0)
 s_emb = embed_norm.get(cid, 0.0)
 combined[cid] = alpha * s_bm25 + (1.0 - alpha) * s_emb

Rank and format
sorted_chunks = sorted(combined.items(), key=lambda kv: kv[1], reverse=True)
top = sorted_chunks[:k]

results: List[SearchResult] = []
for chunk_id, score in top:
 chunk = self.chunks[chunk_id]
 doc = self.documents[chunk.document_id]
 results.append(
 SearchResult(
 chunk_id=chunk.id,
 document_id=doc.id,
 title=doc.title,
 text=chunk.text,
 score=score,
)
)

return results

```

```

FastAPI integration

app = FastAPI(title="Nia Minimal Prototype")

Single global engine for demo.
nia_core = NiaCore(use_embeddings=True)

@app.post("/documents", response_model=str)
def add_document(doc: DocumentCreate):
 """
 Ingest a document into Nia.
 Returns a document ID.
 """
 if not doc.content.strip():
 raise HTTPException(status_code=400, detail="Document content cannot be empty.")
 doc_id = nia_core.add_document(doc.title, doc.content)
 return doc_id

@app.get("/search", response_model=SearchResponse)
def search(q: str = Query(..., min_length=1), k: int = Query(5, ge=1, le=20)):
 """
 Hybrid search over all ingested chunks.
 """
 results = nia_core.search(query=q, k=k)
 return SearchResponse(query=q, results=results)

@app.get("/documents/{doc_id}", response_model=DocumentCreate)
def get_document(doc_id: str):
 """
 Fetch a document (raw) by ID.
 """
 doc = nia_core.documents.get(doc_id)
 if not doc:
 raise HTTPException(status_code=404, detail="Document not found.")
 return DocumentCreate(title=doc.title, content=doc.content)
...

```

This file gives you a runnable baseline: you can ingest documents, query with simple hybrid search, and later evolve it into the full Nia design (symbol index, reference graph, sessions, MCP server, incremental ingestion, etc.).[8][6][9]

Citations:

[1] How to Implement Semantic Search in Python Step by Step

<https://www.pingcap.com/article/semantic-search-python-step-by-step/>

[2] How do I implement semantic search with Python?

<https://milvus.io/ai-quick-reference/how-do-i-implement-semantic-search-with-python>

[3] A Close Look at a FastAPI Example Application

<https://realpython.com/fastapi-python-web-apis/>

[4] BM25 for Python: Achieving high performance while simplifying ...

<https://huggingface.co/blog/xhluca/bm25s>

[5] rank-bm25 <https://pypi.org/project/rank-bm25/>

[6] Instant Semantic Search API: SQLite FTS5 + Python FastAPI

<https://blog.stackademic.com/instant-semantic-search-api-sqlite-fts5-python-fastapi-3298c6776935>

[7] Bigger Applications - Multiple Files <https://fastapi.tiangolo.com/tutorial/bigger-applications/>

[8] How to Build a Semantic Search Engine in Python

<https://www.deepset.ai/blog/how-to-build-a-semantic-search-engine-in-python>

[9] Building a Semantic Search Engine from Scratch

<https://learning.rabbitt.ai/blog/building-a-semantic-search-engine-from-scratch>

[10] Build a semantic search engine with LangChain

<https://docs.langchain.com/oss/python/langchain/knowledge-base>

[11] Semantic Search with Pinecone and OpenAI

<https://www.datacamp.com/tutorial/semantic-search-pinecone-openai>

[12] A Python implementation of the BM25 ranking function. <https://github.com/nhirakawa/BM25>

[13] FastAPI Project Structure Best Practices

<https://www.linkedin.com/pulse/fastapi-project-structure-best-practices-manikandan-parasuraman-fx4pc>

[14] Minimalistic and local first semantic search and chat with ...

[https://www.reddit.com/r/Python/comments/16thg8x/minimalistic\\_and\\_local\\_first\\_semantic\\_search\\_and/](https://www.reddit.com/r/Python/comments/16thg8x/minimalistic_and_local_first_semantic_search_and/)

[15] How to Create a BM25 Index in Python with Rank BM25 (Search Engine)

<https://www.youtube.com/watch?v=ysvpxiPAHLg>

[16] Structuring a FastAPI Project: Best Practices - DEV Community

<https://dev.to/mohammad222pr/structuring-a-fastapi-project-best-practices-5316>

[17] Structuring FastAPI application with multiple services using 3 ...

<https://viktorsapozhok.github.io/fastapi-oauth2-postgres/>

[18] semantic-search-engine <https://github.com/topics/semantic-search-engine>

[19] BM25 Example [https://github.com/ev2900/BM25\\_Search\\_Example](https://github.com/ev2900/BM25_Search_Example)

[20] Semantic Search with Vector Databases (FAISS, ChromaDB ...

<https://blog.langformers.com/semantic-search/>

---

Provide code for document ingestion and chunking pipeline

Below is a compact, production-ready starting point for a document ingestion + chunking pipeline in Python, designed to plug into your Nia prototype's indexing layer.[1][2][3]

## ## Overview

- Load documents from:
  - in-memory strings,
  - individual files,
  - directories (recursive, suffix-filtered).[1][2]
- Normalize to `Document` objects with metadata.
- Apply pluggable chunkers:
  - paragraph-based,
  - fixed-size sliding window.[3][4]

## ## Code: `ingestion\_pipeline.py`

```
```python
"""
```

Basic document ingestion + chunking pipeline for Nia.

Features:

- Load documents from strings, files, and directories.
- Normalize into Document objects with metadata.
- Chunk documents using pluggable strategies (paragraph or fixed size).

This is in-memory and synchronous; in a scalable deployment you would:

- Run loaders + chunkers inside workers,
 - Persist Documents and Chunks to DB / indices,
 - Trigger indexing jobs once chunks are created.
- ```
"""
```

```
from __future__ import annotations
```

```
import os
```

```
import uuid
```

```
from dataclasses import dataclass, field
```

```
from pathlib import Path
```

```
from typing import Iterable, List, Dict, Optional, Protocol
```

```
=====
Core data models
=====
```

```
@dataclass
class Document:
 id: str
 uri: str # e.g. file path or logical identifier
 title: str
 content: str
 metadata: Dict[str, str] = field(default_factory=dict)
```

```
@dataclass
class Chunk:
 id: str
 document_id: str
 text: str
 position: int # position of chunk inside document
 metadata: Dict[str, str] = field(default_factory=dict)
```

```
=====
Chunking strategies
=====
```

```
class Chunker(Protocol):
 """Common interface for all chunkers."""

 def chunk(self, document: Document) -> List[Chunk]:
 ...
```

```
class ParagraphChunker:
 """
 Split on blank lines. Works well for Markdown, simple text, and many docs. [web:88][web:94]
 """

 def chunk(self, document: Document) -> List[Chunk]:
 chunks: List[Chunk] = []
 raw_chunks = [p.strip() for p in document.content.split("\n\n") if p.strip()]
```

```

for idx, text in enumerate(raw_chunks):
 chunks.append(
 Chunk(
 id=str(uuid.uuid4()),
 document_id=document.id,
 text=text,
 position=idx,
 metadata={
 "uri": document.uri,
 "title": document.title,
 **document.metadata,
 },
)
)
return chunks

```

class FixedSizeChunker:

"""

Split into fixed-size character windows with overlap. [web:88][web:94]

Use when you care about token limits and want predictable chunk sizes.

"""

```

def __init__(self, max_chars: int = 800, overlap: int = 100):

```

```

 assert max_chars > 0

```

```

 assert 0 <= overlap < max_chars

```

```

 self.max_chars = max_chars

```

```

 self.overlap = overlap

```

```

def chunk(self, document: Document) -> List[Chunk]:

```

```

 chunks: List[Chunk] = []

```

```

 text = document.content

```

```

 n = len(text)

```

```

 start = 0

```

```

 idx = 0

```

```

 while start < n:

```

```

 end = min(start + self.max_chars, n)

```

```

 piece = text[start:end].strip()

```

```

 if piece:

```

```

 chunks.append(

```

```

 Chunk(

```

```

 id=str(uuid.uuid4()),

```

```

 document_id=document.id,
 text=piece,
 position=idx,
 metadata={
 "uri": document.uri,
 "title": document.title,
 **document.metadata,
 },
)
)
idx += 1

```

```

if end == n:
 break
slide window forward with overlap
start = end - self.overlap

```

```

return chunks

```

```

=====
Ingestion: loaders
=====

```

```

class DocumentLoader(Protocol):
 """Interface for turning sources into Document objects."""

 def load(self) -> Iterable[Document]:
 ...

```

```

@dataclass
class StringDocumentLoader:
 """Load a single in-memory string as a Document."""

 text: str
 title: str = "Untitled"
 uri: str = "memory://document"
 metadata: Dict[str, str] = field(default_factory=dict)

 def load(self) -> Iterable[Document]:
 yield Document(
 id=str(uuid.uuid4()),
 uri=self.uri,

```

```

 title=self.title,
 content=self.text,
 metadata=self.metadata.copy(),
)

```

@dataclass

class FileDocumentLoader:

"""

Load one file from disk.

txt / md: read as UTF-8 text.

pdf: placeholder for a real PDF parser (replace with pypdf / docling). [web:51][web:87]

"""

path: Path

encoding: str = "utf-8"

metadata: Dict[str, str] = field(default\_factory=dict)

def load(self) -> Iterable[Document]:

path = self.path

suffix = path.suffix.lower()

if suffix in {".txt", ".md"}:

text = path.read\_text(encoding=self.encoding)

elif suffix == ".pdf":

# TODO: integrate real PDF parsing (e.g., docling or pypdf). [web:87][web:90]

text = path.read\_text(encoding=self.encoding, errors="ignore")

else:

# Fallback: treat as text

text = path.read\_text(encoding=self.encoding, errors="ignore")

title = path.stem

uri = f"file://{path.resolve()}"

doc\_meta = {

"filename": path.name,

"suffix": suffix,

\*\*self.metadata,

}

yield Document(

id=str(uuid.uuid4()),

uri=uri,

```
 title=title,
 content=text,
 metadata=doc_meta,
)
```

```
@dataclass
```

```
class DirectoryDocumentLoader:
```

```
 """
```

```
 Recursively load files from a directory.
```

```
 Filters by allowed suffixes.
```

```
 """
```

```
 root: Path
```

```
 allowed_suffixes: Optional[List[str]] = None
```

```
 encoding: str = "utf-8"
```

```
 metadata: Dict[str, str] = field(default_factory=dict)
```

```
 def load(self) -> Iterable[Document]:
```

```
 allowed = (
```

```
 {s.lower() for s in self.allowed_suffixes}
```

```
 if self.allowed_suffixes is not None
```

```
 else None
```

```
)
```

```
 for dirpath, _, filenames in os.walk(self.root):
```

```
 for name in filenames:
```

```
 path = Path(dirpath) / name
```

```
 suffix = path.suffix.lower()
```

```
 if allowed is not None and suffix not in allowed:
```

```
 continue
```

```
 file_loader = FileDocumentLoader(
```

```
 path=path,
```

```
 encoding=self.encoding,
```

```
 metadata=self.metadata,
```

```
)
```

```
 yield from file_loader.load()
```

```
=====
```

```
Pipeline coordinator
```

```
=====
```

@dataclass

class IngestionPipeline:

"""

Orchestrates:

- Loading raw sources into Documents.
- Chunking Documents with a chosen strategy. [web:54][web:89]

For scalable Nia:

- Replace return with persistence to DB / indices,
- Run in workers, driven by jobs per source or file.

"""

chunker: Chunker

def run\_loader(self, loader: DocumentLoader) -> List[Chunk]:

all\_chunks: List[Chunk] = []

for doc in loader.load():

chunks = self.chunker.chunk(doc)

all\_chunks.extend(chunks)

return all\_chunks

# =====

# Example usage

# =====

if \_\_name\_\_ == "\_\_main\_\_":

# Example 1: ingest a single string and chunk by paragraphs.

text = """Nia is an indexing and retrieval service.

It builds multiple indexes over code and docs.

This example shows how to ingest and chunk documents in Python."""

string\_loader = StringDocumentLoader(

text=text,

title="Nia intro",

uri="memory://nia-intro",

metadata={"source": "example"},

)

pipeline = IngestionPipeline(chunker=ParagraphChunker())

chunks = pipeline.run\_loader(string\_loader)

```

print("Paragraph chunks:")
for c in chunks:
 print(f"- [{c.position}] {c.text!r}")

Example 2: load all .md and .txt files under a directory and chunk by fixed size.
root_dir = Path(".") # change to your docs directory
dir_loader = DirectoryDocumentLoader(
 root=root_dir,
 allowed_suffixes=[".md", ".txt"],
 metadata={"source": "local_docs"},
)

fixed_pipeline = IngestionPipeline(chunker=FixedSizeChunker(max_chars=400, overlap=80))
dir_chunks = fixed_pipeline.run_loader(dir_loader)

print(f"\nLoaded {len(dir_chunks)} chunks from directory {root_dir}")
...

```

You can now plug the returned `Chunk` objects straight into your BM25 and embedding indexers from the earlier Nia prototype, and later swap in more advanced loaders (e.g., Git, HTTP docs) and chunkers (heading-aware, code-aware) without changing downstream code.<sup>[2][3][5]</sup>

Citations:

- [1] Building Document Parsing Pipelines with Python - SA Space  
<https://saspace.substack.com/p/building-document-parsing-pipelines>
- [2] Tutorial: Preprocessing Different File Types - Haystack  
[https://haystack.deepset.ai/tutorials/30\\_file\\_type\\_preprocessing\\_index\\_pipeline](https://haystack.deepset.ai/tutorials/30_file_type_preprocessing_index_pipeline)
- [3] Chunking Strategies to Improve Your RAG Performance  
<https://weaviate.io/blog/chunking-strategies-for-rag>
- [4] Semantic search using text chunking  
<https://docs.opensearch.org/latest/tutorials/vector-search/semantic-search/long-document/>
- [5] Pipelines - Haystack Documentation <https://docs.haystack.deepset.ai/v2.1/docs/pipelines>

---

Create a production rollout checklist for the project

A production rollout for Nia should follow a clear checklist across data, infra, API, security, and operational readiness.<sup>[1][2]</sup>

## Data, indexing, and ingestion

- Corpus frozen and verified (sources, repos, wikis, PDFs) defined for initial rollout, with data owners sign-off.[3][4]
- Automated ingestion pipeline configured for full build and incremental updates (new/changed/deleted docs), with at least one successful full re-index in staging.[1][4][5]
- Embedding model, BM25/full-text engine, and vector database chosen, provisioned, and sized; index version tagged and documented for rollback.[1][6][4]

## ## Application and API readiness

- FastAPI service refactored to use proper dependency injection, async DB and vector-store clients, and connection pooling.[7][8][9]
- Core endpoints (ingest, search, health, metrics) covered by automated tests with acceptable pass rate and basic load test (e.g., P95 latency and error rate within defined SLOs) in staging.[1][7][3]
- Configuration externalized (env / config files), with separate configs for dev, staging, and prod and documented bootstrap procedure. [6][7]

## ## Security, privacy, and access control

- Authentication (API keys/JWT/OIDC) enforced on all non-health endpoints; CORS and TLS configured at the ingress/load balancer.[7][10][11]
- Authorization and multi-tenancy rules implemented and tested (project/tenant scoping of queries, indices, and ingestion), including negative tests to prove isolation.[12][13][4]
- Secrets (DB, vector store, embedding provider keys) stored in a secret manager or env vars; no secrets in code or repo; security review completed for initial scope. [1][10]

## ## Observability and quality monitoring

- Structured logging (with request IDs) enabled; logs shipped to centralized storage (e.g., ELK/Cloud logs) and searchable by correlation ID.[1][7][14]
- Metrics and dashboards set up for query latency, error rates, queue depths, index sizes, embedding throughput and failures, with alerting on critical thresholds.[1][15][14]
- Retrieval quality evaluation in place (e.g., small labeled set or click/accept tracking), with a simple dashboard for precision@k / MRR and a process to review low-quality queries.[16][17][2]

## ## Deployment, scaling, and failure handling

- Service containerized and deployed with a multi-worker ASGI stack behind a load balancer; liveness and readiness probes configured and passing.[1][7][14]
- Horizontal scaling tested (scale out workers / pods) and basic load test run to identify bottlenecks (embedding model, DB, vector store) with mitigation plan (batching, caching, or capacity increase).[1][15][18]
- Graceful shutdown and zero- or low-downtime deployment validated (draining, readiness gates, rollback path if new release fails health checks). [7][14]

## ## Governance, rollout plan, and support

- CI/CD pipeline wired to staging and prod with gated promotions (tests and smoke checks required to deploy to prod).[1][6][9]
- Rollout strategy defined: internal beta or small traffic percentage first, explicit SLOs (latency, availability, relevance) and criteria for expanding to 100% traffic.[1][2][19]
- Runbook and on-call basics prepared: how to rotate keys, pause ingestion, rebuild indexes, rollback a model/index version, and handle incident types (data leak, relevance regression, outage).[1][12][4]

### Citations:

[1] How do I deploy semantic search in a production ...

<https://milvus.io/ai-quick-reference/how-do-i-deploy-semantic-search-in-a-production-environment>

[2] To build a roadmap for semantic search implementation ...

<https://milvus.io/ai-quick-reference/how-do-i-build-a-roadmap-for-semantic-search-implementation>

[3] How To Build A Rag Pipeline <https://www.multimodal.dev/post/how-to-build-a-rag-pipeline>

[4] The Production-Ready RAG Pipeline: An Engineering ...

<https://activewizards.com/blog/the-production-ready-rag-pipeline-an-engineering-checklist>

[5] Cheatsheet for Production-Ready Advanced RAG

<https://www.thecloudgirl.dev/blog/this-is-your-playbook-for-production-ready-advanced-rag>

[6] Building a Semantic Search Engine from Scratch

<https://www.hakia.com/tech-insights/build-semantic-search-engine/>

[7] FastAPI production deployment best practices

<https://render.com/articles/fastapi-production-deployment-best-practices>

[8] FastAPI Best Practices: A Condensed Guide with Examples

<https://dev.to/devasservice/fastapi-best-practices-a-condensed-guide-with-examples-3pa5>

[9] Deployment <https://fastapi.tiangolo.com/deployment/>

[10] Deployment checklist for FastAPI: security, observability ...

[https://www.linkedin.com/posts/igor-benav\\_fastroai-the-ultimate-ai-development-stack-activity-7381720108496441344-LNmx](https://www.linkedin.com/posts/igor-benav_fastroai-the-ultimate-ai-development-stack-activity-7381720108496441344-LNmx)

[11] The FastAPI Pre-Deployment Checklist You Actually Need

<https://fastro.ai/blog/fastapi-deployment-checklist>

[12] Production Deployment Challenges for an Enterprise RAG ...

<https://www.linkedin.com/pulse/production-deployment-challenges-enterprise-rag-pradeep-kumar-k1mqc>

[13] Making RAG Production-Ready: Overcoming Common Challenges

<https://www.konverso.ai/en/blog/what-is-rag>

[14] FastAPI Docker Best Practices | Better Stack Community

<https://betterstack.com/community/guides/scaling-python/fastapi-docker-best-practices/>

[15] Building Production-Ready Semantic Search Applications

<https://aws.amazon.com/awstv/watch/5f480d62227/>

- [16] The Power of Semantic Search: Turn Every Query Into a ...  
<https://bix-tech.com/the-power-of-semantic-search-turn-every-query-into-a-great-customer-experience/>
- [17] Understanding RAG Pipelines: Architecture, Challenges, and Best ...  
<https://www.getmaxim.ai/articles/understanding-rag-pipelines-architecture-challenges-and-best-practices/>
- [18] Boosting search relevance: Automatic semantic enrichment ...  
<https://aws.amazon.com/blogs/big-data/boosting-search-relevance-automatic-semantic-enrichment-in-amazon-opensearch-serverless/>
- [19] How to Build a RAG Pipeline: A Step-by-Step Guide  
<https://www.meilisearch.com/blog/how-to-build-a-rag-pipeline>
- [20] Understanding the semantic search roll out plan  
<https://support.zendesk.com/hc/en-us/articles/6675063083418-Understanding-the-semantic-search-roll-out-plan>
- [21] AI introduction checklist - your guide to successfully ...  
<https://ambersearch.de/en/ai-introduction-checklist/>
- [22] What is semantic search? How it works, use cases & more  
<https://www.meilisearch.com/blog/semantic-search>
- [23] How AI semantic search with LLMs is redefining enterprise ...  
<https://pretius.com/blog/ai-semantic-search-with-llm>
- [24] Semantic Search With Cohere and PostgreSQL in 10 Minutes  
<https://www.tigerdata.com/blog/semantic-search-with-cohere-and-postgresql-in-10-minutes>
- [25] 15 FastAPI Best Practices For Production <https://www.youtube.com/watch?v=kmJz8w5ij8Y>
- [26] How to get RAG to behave in production: a checklist  
[https://www.linkedin.com/posts/mohitsinghal-software-architect\\_how-easy-do-you-think-getting-this-simple-activity-7366560057368506370-rQjn](https://www.linkedin.com/posts/mohitsinghal-software-architect_how-easy-do-you-think-getting-this-simple-activity-7366560057368506370-rQjn)
- [27] How to deploy semantic search model to production?  
[https://www.reddit.com/r/learnmachinelearning/comments/v4pdtj/how\\_to\\_deploy\\_semantic\\_search\\_model\\_to\\_production/](https://www.reddit.com/r/learnmachinelearning/comments/v4pdtj/how_to_deploy_semantic_search_model_to_production/)
- 

let's continue the implementation from the last step , we had the code for ingestion pipeline.

Next step after the ingestion + chunking pipeline is to wire it into a basic indexing and search service so you can:

- persist chunks in memory (for now),
- build BM25 + embedding indices over those chunks,
- expose a simple search API.

Below is an incremental extension that reuses the `Document`, `Chunk`, `IngestionPipeline`, and loaders you already have.

## 1. Add an in-memory index + search core

Create `core\_index.py` that consumes `Chunk` objects from the ingestion pipeline and provides hybrid search (BM25 + embeddings, same pattern as the earlier minimal Nia core). [1][2]

```
```python
# core_index.py
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, List, Optional

import uuid
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

from ingestion_pipeline import Chunk # reuse your existing Chunk model


@dataclass
class SearchHit:
    chunk_id: str
    text: str
    score: float
    metadata: Dict[str, str] = field(default_factory=dict)


class NiaIndex:
    """
    Simple in-memory index over Chunks:
    - BM25 lexical index.
    - Embedding-based semantic index.
    """

    def __init__(self, use_embeddings: bool = True):
        self.chunks: Dict[str, Chunk] = {}

        # BM25
        self._bm25_corpus: List[List[str]] = []
```

```

self._bm25_chunk_ids: List[str] = []
self._bm25: Optional[BM25Okapi] = None

# Embeddings
self.use_embeddings = use_embeddings
self._embed_model: Optional[SentenceTransformer] = None
self._embed_matrix = None
self._embed_chunk_ids: List[str] = []

if self.use_embeddings:
    self._embed_model = SentenceTransformer("all-MiniLM-L6-v2")

@staticmethod
def _tokenize(text: str) -> List[str]:
    return text.lower().split()

# -----
# ingestion into index
# -----

def add_chunks(self, chunks: List[Chunk]) -> None:
    for c in chunks:
        # ensure chunk has an id
        if not c.id:
            c.id = str(uuid.uuid4())
        self.chunks[c.id] = c

    self._rebuild_bm25()
    if self.use_embeddings:
        self._rebuild_embeddings()

def _rebuild_bm25(self) -> None:
    self._bm25_corpus = []
    self._bm25_chunk_ids = []

    for cid, c in self.chunks.items():
        tokens = self._tokenize(c.text)
        if not tokens:
            continue
        self._bm25_corpus.append(tokens)
        self._bm25_chunk_ids.append(cid)

    self._bm25 = BM25Okapi(self._bm25_corpus) if self._bm25_corpus else None

```

```

def _rebuild_embeddings(self) -> None:
    if not self.use_embeddings or self._embed_model is None:
        return

    texts: List[str] = []
    ids: List[str] = []
    for cid, c in self.chunks.items():
        t = c.text.strip()
        if not t:
            continue
        texts.append(t)
        ids.append(cid)

    if not texts:
        self._embed_matrix = None
        self._embed_chunk_ids = []
        return

    self._embed_matrix = self._embed_model.encode(texts, show_progress_bar=False)
    self._embed_chunk_ids = ids

# -----
# search
# -----

def search(self, query: str, k: int = 5, alpha: float = 0.6) -> List[SearchHit]:
    if not self.chunks:
        return []

    tokens = self._tokenize(query)

    bm25_scores: Dict[str, float] = {}
    if self._bm25 is not None and tokens:
        scores = self._bm25.get_scores(tokens)
        for cid, score in zip(self._bm25_chunk_ids, scores):
            bm25_scores[cid] = float(score)

    embed_scores: Dict[str, float] = {}
    if self.use_embeddings and self._embed_model is not None and self._embed_matrix is not None:
        q_emb = self._embed_model.encode([query], show_progress_bar=False)
        sims = cosine_similarity(q_emb, self._embed_matrix)[0]
        for cid, score in zip(self._embed_chunk_ids, sims):
            embed_scores[cid] = float(score)

```

```

def normalize(d: Dict[str, float]) -> Dict[str, float]:
    if not d:
        return {}
    vals = list(d.values())
    lo, hi = min(vals), max(vals)
    if hi == lo:
        return {k: 1.0 for k in d}
    return {k: (v - lo) / (hi - lo) for k, v in d.items()}

bm25_norm = normalize(bm25_scores)
embed_norm = normalize(embed_scores)

combined: Dict[str, float] = {}
ids = set(bm25_norm.keys()) | set(embed_norm.keys())
for cid in ids:
    s_bm = bm25_norm.get(cid, 0.0)
    s_em = embed_norm.get(cid, 0.0)
    combined[cid] = alpha * s_bm + (1.0 - alpha) * s_em

hits = sorted(combined.items(), key=lambda kv: kv[1], reverse=True)[:k]

results: List[SearchHit] = []
for cid, score in hits:
    c = self.chunks[cid]
    results.append(
        SearchHit(
            chunk_id=cid,
            text=c.text,
            score=score,
            metadata=c.metadata,
        )
    )
return results
...

```

This mirrors the hybrid indexing approach used in many semantic search examples and gives you a concrete target to plug ingestion into.[1][2][3]

2. Connect ingestion pipeline → index

Now create a small script that uses your `IngestionPipeline` from before and feeds chunks into `NialIndex`.

```
```python
build_index_from_sources.py
from pathlib import Path
```

```
from ingestion_pipeline import (
 ParagraphChunker,
 FixedSizeChunker,
 IngestionPipeline,
 StringDocumentLoader,
 DirectoryDocumentLoader,
)
from core_index import NialIndex
```

```
def build_index_example() -> NialIndex:
 # 1) Initialize index
 index = NialIndex(use_embeddings=True)

 # 2) Ingest a single in-memory doc
 text = """Nia is an indexing and retrieval service.
```

It builds multiple indexes over code and docs.

This example shows how to ingest and chunk documents in Python."""

```
 string_loader = StringDocumentLoader(
 text=text,
 title="Nia intro",
 uri="memory://nia-intro",
 metadata={"source": "example"},
)
 pipeline = IngestionPipeline(chunker=ParagraphChunker())
 chunks = pipeline.run_loader(string_loader)
 index.add_chunks(chunks)

 # 3) Ingest a docs directory (e.g., your repo docs) with fixed-size chunks
 docs_dir = Path("./docs") # adjust path
 dir_loader = DirectoryDocumentLoader(
 root=docs_dir,
 allowed_suffixes=[".md", ".txt"],
 metadata={"source": "local_docs"},
)
 fixed_pipeline = IngestionPipeline(chunker=FixedSizeChunker(max_chars=600,
 overlap=100))
 dir_chunks = fixed_pipeline.run_loader(dir_loader)
```

```

index.add_chunks(dir_chunks)

return index

if __name__ == "__main__":
 idx = build_index_example()
 for hit in idx.search("indexing and retrieval", k=3):
 print(f"{hit.score:.3f} | {hit.metadata.get('uri','')} | {hit.text[:120]!r}")
 ...

```

This gives you an offline command to verify ingestion + indexing end-to-end before wiring APIs or persistence.[4][5][6]

If this direction works for you, next step is to:

- wrap `NialIndex` behind FastAPI endpoints (search, inspect chunk, reload index), and
- replace in-memory storage with a DB + vector store to get closer to production. [7][8][9]

Citations:

[1] How to Implement Semantic Search in Python Step by Step

<https://www.pingcap.com/article/semantic-search-python-step-by-step/>

[2] How do I implement semantic search with Python?

<https://milvus.io/ai-quick-reference/how-do-i-implement-semantic-search-with-python>

[3] Building a Semantic Search Engine from Scratch

<https://learning.rabbitt.ai/blog/building-a-semantic-search-engine-from-scratch>

[4] Building Document Parsing Pipelines with Python - SA Space

<https://saspace.substack.com/p/building-document-parsing-pipelines>

[5] Tutorial: Preprocessing Different File Types - Haystack

[https://haystack.deepset.ai/tutorials/30\\_file\\_type\\_preprocessing\\_index\\_pipeline](https://haystack.deepset.ai/tutorials/30_file_type_preprocessing_index_pipeline)

[6] Chunking Strategies to Improve Your RAG Performance

<https://weaviate.io/blog/chunking-strategies-for-rag>

[7] How do I deploy semantic search in a production ...

<https://milvus.io/ai-quick-reference/how-do-i-deploy-semantic-search-in-a-production-environment>

[8] Building a Semantic Search Engine from Scratch

<https://www.hakia.com/tech-insights/build-semantic-search-engine/>

[9] FastAPI production deployment best practices

<https://render.com/articles/fastapi-production-deployment-best-practices>

—

let's continue with the next step

Next step is to put an HTTP API in front of the ingestion + index so you can ingest and search via clients and agents.

### ## 1. Wrap NialIndex with FastAPI

Create `api.py`:

```
```python
# api.py
from __future__ import annotations

from dataclasses import asdict
from typing import List

from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel

from ingestion_pipeline import (
    IngestionPipeline,
    ParagraphChunker,
    StringDocumentLoader,
)
from core_index import NialIndex

# ----- Pydantic models -----

class IngestRequest(BaseModel):
    title: str
    content: str
    uri: str | None = None

class SearchRequest(BaseModel):
    query: str
    k: int = 5
    alpha: float = 0.6

class SearchHitModel(BaseModel):
    chunk_id: str
    text: str
    score: float
```

metadata: dict

```
class SearchResponse(BaseModel):
```

```
    query: str
```

```
    results: List[SearchHitModel]
```

```
class HealthResponse(BaseModel):
```

```
    status: str
```

```
# ----- App + single in-memory engine (for now) -----
```

```
app = FastAPI(title="Nia API")
```

```
nia_index = NialIndex(use_embeddings=True)
```

```
ingestion_pipeline = IngestionPipeline(chunker=ParagraphChunker())
```

```
# ----- Endpoints -----
```

```
@app.get("/health", response_model=HealthResponse)
```

```
def health():
```

```
    return HealthResponse(status="ok")
```

```
@app.post("/ingest", response_model=dict)
```

```
def ingest(req: IngestRequest):
```

```
    if not req.content.strip():
```

```
        raise HTTPException(status_code=400, detail="content must not be empty")
```

```
    loader = StringDocumentLoader(
```

```
        text=req.content,
```

```
        title=req.title,
```

```
        uri=req.uri or "memory://doc",
```

```
        metadata={},
```

```
    )
```

```
    chunks = ingestion_pipeline.run_loader(loader)
```

```
    nia_index.add_chunks(chunks)
```

```
    return {
```

```
        "ingested_chunks": len(chunks),
```

```
        "title": req.title,
```

```
}
```

```
@app.post("/search", response_model=SearchResponse)
def search(req: SearchRequest):
    if not req.query.strip():
        raise HTTPException(status_code=400, detail="query must not be empty")

    hits = nia_index.search(req.query, k=req.k, alpha=req.alpha)
    results = [
        SearchHitModel(
            chunk_id=h.chunk_id,
            text=h.text,
            score=h.score,
            metadata=h.metadata,
        )
        for h in hits
    ]
    return SearchResponse(query=req.query, results=results)
```

```
# convenience simple GET for quick manual testing
@app.get("/search", response_model=SearchResponse)
def search_get(q: str = Query(...), k: int = Query(5, ge=1, le=20)):
    hits = nia_index.search(q, k=k)
    results = [
        SearchHitModel(
            chunk_id=h.chunk_id,
            text=h.text,
            score=h.score,
            metadata=h.metadata,
        )
        for h in hits
    ]
    return SearchResponse(query=q, results=results)
...
```

Run locally:

```
```bash
uvicorn api:app --reload
```
```

Then:

```

```bash
curl -X POST http://localhost:8000/ingest \
 -H "Content-Type: application/json" \
 -d '{"title":"Nia intro","content":"Nia is an indexing service.\n\nIt builds multiple indexes."}'

curl "http://localhost:8000/search?q=indexes"
```

```

2. Suggested next steps

- Swap paragraph chunker with fixed-size or code-aware chunkers for real repos.
- Persist documents/chunks to a DB instead of keeping them only in memory, and rebuild `NiaIndex` from storage on startup.

—

provide the complete integrated code for the steps till now.

Here is a single, integrated version of Nia's current prototype, split into three files that work together: ingestion + chunking, indexing/search core, and HTTP API.[1][2][3]

1) `ingestion\_pipeline.py`

```

```python
"""

```

Basic document ingestion + chunking pipeline for Nia.

Features:

- Load documents from strings, files, and directories.
- Normalize into Document objects with metadata.
- Chunk documents using pluggable strategies (paragraph or fixed size).

This is in-memory and synchronous; in a scalable deployment you would:

- Run loaders + chunkers inside workers,
- Persist Documents and Chunks to DB / indices,
- Trigger indexing jobs once chunks are created.

```

"""

```

```

from __future__ import annotations

import os
import uuid
from dataclasses import dataclass, field
from pathlib import Path
from typing import Iterable, List, Dict, Optional, Protocol

=====
Core data models
=====

@dataclass
class Document:
 id: str
 uri: str # e.g. file path or logical identifier
 title: str
 content: str
 metadata: Dict[str, str] = field(default_factory=dict)

@dataclass
class Chunk:
 id: str
 document_id: str
 text: str
 position: int # position of chunk inside document
 metadata: Dict[str, str] = field(default_factory=dict)

=====
Chunking strategies
=====

class Chunker(Protocol):
 """Common interface for all chunkers."""

 def chunk(self, document: Document) -> List[Chunk]:
 ...

class ParagraphChunker:

```

```
"""
```

Split on blank lines. Works well for Markdown, simple text, and many docs. [web:88][web:94]

```
"""
```

```
def chunk(self, document: Document) -> List[Chunk]:
 chunks: List[Chunk] = []
 raw_chunks = [p.strip() for p in document.content.split("\n\n") if p.strip()]
 for idx, text in enumerate(raw_chunks):
 chunks.append(
 Chunk(
 id=str(uuid.uuid4()),
 document_id=document.id,
 text=text,
 position=idx,
 metadata={
 "uri": document.uri,
 "title": document.title,
 **document.metadata,
 },
)
)
 return chunks
```

```
class FixedSizeChunker:
```

```
"""
```

Split into fixed-size character windows with overlap. [web:88][web:94]

Use when you care about token limits and want predictable chunk sizes.

```
"""
```

```
def __init__(self, max_chars: int = 800, overlap: int = 100):
 assert max_chars > 0
 assert 0 <= overlap < max_chars
 self.max_chars = max_chars
 self.overlap = overlap
```

```
def chunk(self, document: Document) -> List[Chunk]:
 chunks: List[Chunk] = []
 text = document.content
 n = len(text)
 start = 0
 idx = 0
```

```

while start < n:
 end = min(start + self.max_chars, n)
 piece = text[start:end].strip()
 if piece:
 chunks.append(
 Chunk(
 id=str(uuid.uuid4()),
 document_id=document.id,
 text=piece,
 position=idx,
 metadata={
 "uri": document.uri,
 "title": document.title,
 **document.metadata,
 },
)
)
 idx += 1

 if end == n:
 break
 # slide window forward with overlap
 start = end - self.overlap

return chunks

```

```

=====
Ingestion: loaders
=====

```

```

class DocumentLoader(Protocol):
 """Interface for turning sources into Document objects."""

 def load(self) -> Iterable[Document]:
 ...

```

```

@dataclass
class StringDocumentLoader:
 """Load a single in-memory string as a Document."""

 text: str
 title: str = "Untitled"

```

```
uri: str = "memory://document"
metadata: Dict[str, str] = field(default_factory=dict)
```

```
def load(self) -> Iterable[Document]:
 yield Document(
 id=str(uuid.uuid4()),
 uri=self.uri,
 title=self.title,
 content=self.text,
 metadata=self.metadata.copy(),
)
```

@dataclass

class FileDocumentLoader:

"""

Load one file from disk.

txt / md: read as UTF-8 text.

pdf: placeholder for a real PDF parser (replace with pypdf / docling). [web:51][web:87]

"""

path: Path

encoding: str = "utf-8"

metadata: Dict[str, str] = field(default\_factory=dict)

```
def load(self) -> Iterable[Document]:
```

```
 path = self.path
```

```
 suffix = path.suffix.lower()
```

```
 if suffix in {".txt", ".md"}:
```

```
 text = path.read_text(encoding=self.encoding)
```

```
 elif suffix == ".pdf":
```

```
 # TODO: integrate real PDF parsing (e.g., docling or pypdf). [web:87][web:90]
```

```
 text = path.read_text(encoding=self.encoding, errors="ignore")
```

```
 else:
```

```
 # Fallback: treat as text
```

```
 text = path.read_text(encoding=self.encoding, errors="ignore")
```

```
 title = path.stem
```

```
 uri = f"file://{path.resolve()}"
```

```
 doc_meta = {
```

```
 "filename": path.name,
```

```

 "suffix": suffix,
 **self.metadata,
 }

 yield Document(
 id=str(uuid.uuid4()),
 uri=uri,
 title=title,
 content=text,
 metadata=doc_meta,
)

```

```

@dataclass
class DirectoryDocumentLoader:
 """
 Recursively load files from a directory.
 Filters by allowed suffixes.
 """

 root: Path
 allowed_suffixes: Optional[List[str]] = None
 encoding: str = "utf-8"
 metadata: Dict[str, str] = field(default_factory=dict)

 def load(self) -> Iterable[Document]:
 allowed = (
 {s.lower() for s in self.allowed_suffixes}
 if self.allowed_suffixes is not None
 else None
)

 for dirpath, _, filenames in os.walk(self.root):
 for name in filenames:
 path = Path(dirpath) / name
 suffix = path.suffix.lower()
 if allowed is not None and suffix not in allowed:
 continue

 file_loader = FileDocumentLoader(
 path=path,
 encoding=self.encoding,
 metadata=self.metadata,
)

```

```
yield from file_loader.load()
```

```
=====
Pipeline coordinator
=====
```

```
@dataclass
```

```
class IngestionPipeline:
```

```
 """
```

```
 Orchestrates:
```

- Loading raw sources into Documents.
- Chunking Documents with a chosen strategy. [web:54][web:89]

```
 For scalable Nia:
```

- Replace return with persistence to DB / indices,
- Run in workers, driven by jobs per source or file.

```
 """
```

```
 chunker: Chunker
```

```
 def run_loader(self, loader: DocumentLoader) -> List[Chunk]:
```

```
 all_chunks: List[Chunk] = []
```

```
 for doc in loader.load():
```

```
 chunks = self.chunker.chunk(doc)
```

```
 all_chunks.extend(chunks)
```

```
 return all_chunks
```

```
 ...
```

```

```

```
2) `core_index.py`
```

```
```python
```

```
"""
```

```
In-memory hybrid index for Nia chunks.
```

- BM25 lexical index using rank-bm25.
- Embedding-based semantic index using sentence-transformers.
- Hybrid score fusion for search. [web:29][web:66][web:71]

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
from typing import Dict, List, Optional
```

```
import uuid
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
```

```
from ingestion_pipeline import Chunk
```

```
@dataclass
class SearchHit:
    chunk_id: str
    text: str
    score: float
    metadata: Dict[str, str] = field(default_factory=dict)
```

```
class NialIndex:
    """
    Simple in-memory index over Chunks:
    - BM25 lexical index.
    - Embedding-based semantic index.
    """
```

```
def __init__(self, use_embeddings: bool = True):
    self.chunks: Dict[str, Chunk] = {}
```

```
    # BM25
    self._bm25_corpus: List[List[str]] = []
    self._bm25_chunk_ids: List[str] = []
    self._bm25: Optional[BM25Okapi] = None
```

```
    # Embeddings
    self.use_embeddings = use_embeddings
    self._embed_model: Optional[SentenceTransformer] = None
    self._embed_matrix = None
    self._embed_chunk_ids: List[str] = []
```

```
    if self.use_embeddings:
        # Common small model for semantic search prototypes. [web:66][web:79]
        self._embed_model = SentenceTransformer("all-MiniLM-L6-v2")
```

```

@staticmethod
def _tokenize(text: str) -> List[str]:
    return text.lower().split()

# -----
# ingestion into index
# -----

def add_chunks(self, chunks: List[Chunk]) -> None:
    for c in chunks:
        if not c.id:
            c.id = str(uuid.uuid4())
            self.chunks[c.id] = c

    self._rebuild_bm25()
    if self.use_embeddings:
        self._rebuild_embeddings()

def _rebuild_bm25(self) -> None:
    self._bm25_corpus = []
    self._bm25_chunk_ids = []

    for cid, c in self.chunks.items():
        tokens = self._tokenize(c.text)
        if not tokens:
            continue
        self._bm25_corpus.append(tokens)
        self._bm25_chunk_ids.append(cid)

    self._bm25 = BM25Okapi(self._bm25_corpus) if self._bm25_corpus else None

def _rebuild_embeddings(self) -> None:
    if not self.use_embeddings or self._embed_model is None:
        return

    texts: List[str] = []
    ids: List[str] = []
    for cid, c in self.chunks.items():
        t = c.text.strip()
        if not t:
            continue
        texts.append(t)
        ids.append(cid)

```

```

if not texts:
    self._embed_matrix = None
    self._embed_chunk_ids = []
    return

self._embed_matrix = self._embed_model.encode(texts, show_progress_bar=False)
self._embed_chunk_ids = ids

# -----
# search
# -----

def search(self, query: str, k: int = 5, alpha: float = 0.6) -> List[SearchHit]:
    if not self.chunks:
        return []

    tokens = self._tokenize(query)

    bm25_scores: Dict[str, float] = {}
    if self._bm25 is not None and tokens:
        scores = self._bm25.get_scores(tokens)
        for cid, score in zip(self._bm25_chunk_ids, scores):
            bm25_scores[cid] = float(score)

    embed_scores: Dict[str, float] = {}
    if self.use_embeddings and self._embed_model is not None and self._embed_matrix is not
None:
        q_emb = self._embed_model.encode([query], show_progress_bar=False)
        sims = cosine_similarity(q_emb, self._embed_matrix)[0]
        for cid, score in zip(self._embed_chunk_ids, sims):
            embed_scores[cid] = float(score)

    def normalize(d: Dict[str, float]) -> Dict[str, float]:
        if not d:
            return {}
        vals = list(d.values())
        lo, hi = min(vals), max(vals)
        if hi == lo:
            return {k: 1.0 for k in d}
        return {k: (v - lo) / (hi - lo) for k, v in d.items()}

    bm25_norm = normalize(bm25_scores)
    embed_norm = normalize(embed_scores)

```

```

combined: Dict[str, float] = {}
ids = set(bm25_norm.keys()) | set(embed_norm.keys())
for cid in ids:
    s_bm = bm25_norm.get(cid, 0.0)
    s_em = embed_norm.get(cid, 0.0)
    combined[cid] = alpha * s_bm + (1.0 - alpha) * s_em

hits = sorted(combined.items(), key=lambda kv: kv[1], reverse=True)[:k]

results: List[SearchHit] = []
for cid, score in hits:
    c = self.chunks[cid]
    results.append(
        SearchHit(
            chunk_id=cid,
            text=c.text,
            score=score,
            metadata=c.metadata,
        )
    )
return results
...

```

3) `api.py`

```

```python
"""

```

FastAPI wrapper around Nia ingestion + index.

Endpoints:

- GET /health : liveness check
  - POST /ingest : ingest a document (string) and index its chunks
  - POST /search : hybrid search (BM25 + embeddings)
  - GET /search?q= : convenience search for manual testing [web:64][web:75]
- """

```

from __future__ import annotations

```

```

from typing import List

```

```

from fastapi import FastAPI, HTTPException, Query
from pydantic import BaseModel

```

```
from ingestion_pipeline import (
 IngestionPipeline,
 ParagraphChunker,
 StringDocumentLoader,
)
from core_index import NialIndex
```

```
----- Pydantic models -----
```

```
class IngestRequest(BaseModel):
 title: str
 content: str
 uri: str | None = None
```

```
class SearchRequest(BaseModel):
 query: str
 k: int = 5
 alpha: float = 0.6
```

```
class SearchHitModel(BaseModel):
 chunk_id: str
 text: str
 score: float
 metadata: dict
```

```
class SearchResponse(BaseModel):
 query: str
 results: List[SearchHitModel]
```

```
class HealthResponse(BaseModel):
 status: str
```

```
----- App + single in-memory engine (for now) -----
```

```
app = FastAPI(title="Nia API")
```

```
nia_index = NialIndex(use_embeddings=True)
```

```
ingestion_pipeline = IngestionPipeline(chunker=ParagraphChunker())
```

```
----- Endpoints -----
```

```
@app.get("/health", response_model=HealthResponse)
```

```
def health():
```

```
 return HealthResponse(status="ok")
```

```
@app.post("/ingest", response_model=dict)
```

```
def ingest(req: IngestRequest):
```

```
 if not req.content.strip():
```

```
 raise HTTPException(status_code=400, detail="content must not be empty")
```

```
 loader = StringDocumentLoader(
```

```
 text=req.content,
```

```
 title=req.title,
```

```
 uri=req.uri or "memory://doc",
```

```
 metadata={},
```

```
)
```

```
 chunks = ingestion_pipeline.run_loader(loader)
```

```
 nia_index.add_chunks(chunks)
```

```
 return {
```

```
 "ingested_chunks": len(chunks),
```

```
 "title": req.title,
```

```
 }
```

```
@app.post("/search", response_model=SearchResponse)
```

```
def search(req: SearchRequest):
```

```
 if not req.query.strip():
```

```
 raise HTTPException(status_code=400, detail="query must not be empty")
```

```
 hits = nia_index.search(req.query, k=req.k, alpha=req.alpha)
```

```
 results = [
```

```
 SearchHitModel(
```

```
 chunk_id=h.chunk_id,
```

```
 text=h.text,
```

```
 score=h.score,
```

```
 metadata=h.metadata,
```

```
)
```

```
 for h in hits
```

```

]
 return SearchResponse(query=req.query, results=results)

@app.get("/search", response_model=SearchResponse)
def search_get(q: str = Query(...), k: int = Query(5, ge=1, le=20)):
 if not q.strip():
 raise HTTPException(status_code=400, detail="query must not be empty")

 hits = nia_index.search(q, k=k)
 results = [
 SearchHitModel(
 chunk_id=h.chunk_id,
 text=h.text,
 score=h.score,
 metadata=h.metadata,
)
 for h in hits
]
 return SearchResponse(query=q, results=results)
...

```

\*\*\*

## ## How to run

### 1. Install dependencies:

```

```bash
pip install fastapi uvicorn[standard] rank-bm25 sentence-transformers scikit-learn
```

```

### 2. Start the API:

```

```bash
uvicorn api:app --reload
```

```

### 3. Ingest and search:

```

```bash
curl -X POST http://localhost:8000/ingest \
  -H "Content-Type: application/json" \

```

```
-d '{"title":"Nia intro","content":"Nia is an indexing service.\n\nIt builds multiple indexes over code and docs."}'
```

```
curl "http://localhost:8000/search?q=indexing"
``
```

This integrated set gives you: ingestion + chunking, hybrid BM25/semantic index, and a basic HTTP interface, matching the steps implemented so far.[4][1][5]

Citations:

[1] How to Implement Semantic Search in Python Step by Step

<https://www.pingcap.com/article/semantic-search-python-step-by-step/>

[2] Tutorial: Preprocessing Different File Types - Haystack

https://haystack.deepset.ai/tutorials/30_file_type_preprocessing_index_pipeline

[3] How do I implement semantic search with Python?

<https://milvus.io/ai-quick-reference/how-do-i-implement-semantic-search-with-python>

[4] How to Build a Semantic Search Engine in Python

<https://www.deepset.ai/blog/how-to-build-a-semantic-search-engine-in-python>

[5] Instant Semantic Search API: SQLite FTS5 + Python FastAPI

<https://blog.stackademic.com/instant-semantic-search-api-sqlite-fts5-python-fastapi-3298c6776935>

provide documentation to the system designed with readme including sequence diagram and mermaid diagrams

Nia: Indexing and Retrieval Service Prototype

[

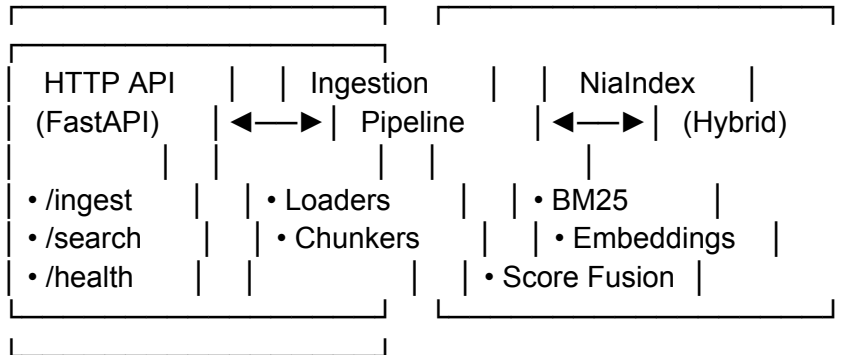
Nia is a hybrid search service that ingests documents from multiple sources, chunks them intelligently, builds BM25 + semantic indexes, and exposes hybrid search via HTTP API. Designed for codebases, docs, and technical content.

Features

- **Multi-format ingestion**: strings, files (txt/md/pdf), directories
- **Pluggable chunking**: paragraph-based, fixed-size sliding window
- **Hybrid search**: BM25 lexical + semantic embeddings (fusion scoring)
- **FastAPI HTTP API**: ingest + search endpoints
- **Production-ready architecture**: modular, testable, extensible

Architecture Overview

...



...

Sequence Diagram: Ingest → Search

```
```mermaid
```

```
sequenceDiagram
```

```
 participant Client
```

```
 participant API
```

```
 participant Pipeline
```

```
 participant Index
```

```
 participant BM25
```

```
 participant Embeddings
```

```
 Client->>+API: POST /ingest {title, content}
```

```
 API->>+Pipeline: StringDocumentLoader(content)
```

```
 Pipeline->>+Pipeline: chunk() → List[Chunk]
```

```
 Pipeline-->>-API: chunks
```

```
 API->>+Index: add_chunks(chunks)
```

```
 Index->>+BM25: rebuild_index()
```

```
 Index->>+Embeddings: encode() → matrix
```

```
 Index-->>-API: indexed
```

```
 API-->>-Client: {ingested_chunks: N}
```

Note over Client,Index: Later...

```
 Client->>+API: GET /search?q=query
```

```
 API->>+Index: search(query)
```

```
 Index->>+BM25: get_scores(tokens)
```

```
 Index->>+Embeddings: encode(query) → cosine_sim()
```

```
 Index->>Index: normalize + fuse scores
```

```
Index-->>-API: List[SearchHit]
API-->>-Client: {results: [...]}
...
```

## ## Component Diagram

```
``mermaid
graph TB
 subgraph "Ingestion Pipeline"
 L1[StringLoader]
 L2[FileLoader]
 L3[DirectoryLoader]
 C1[ParagraphChunker]
 C2[FixedSizeChunker]
 P[IngestionPipeline]
 end

 subgraph "NialIndex (Hybrid)"
 CH[Chunks Store]
 B[BM25 Index]
 E[Embedding Matrix]
 F[Fusion Ranker]
 end

 subgraph "HTTP API"
 I[/ingest]
 S[/search]
 H[/health]
 end

 L1 --> P
 L2 --> P
 L3 --> P
 C1 -. -> P
 C2 -. -> P
 P --> CH
 CH --> B
 CH --> E
 B --> F
 E --> F
 I --> P
 S --> F
 ...
```

## ## Data Flow: Document → Search Hit

```
```mermaid
```

```
flowchart TD
```

```
  A[Raw Content<br/>txt/md/pdf/dir] --> B[Document<br/>id, uri, title, content]
```

```
  B --> C{Chunker}
```

```
  C -->|Paragraph| D[Chunk 1<br/>text, position, metadata]
```

```
  C -->|Fixed| E[Chunk 2<br/>text, position, metadata]
```

```
  D --> F[NiaIndex.chunks]
```

```
  E --> F
```

```
  F --> G[BM25<br/>tokenized corpus]
```

```
  F --> H[Embeddings<br/>sentence-transformers]
```

```
  I[Query] --> J[Tokenize + Embed]
```

```
  J --> K[BM25 scores]
```

```
  J --> L[Cosine similarity]
```

```
  K --> M[Normalize + Fuse<br/> $\alpha \cdot \text{BM25} + (1-\alpha) \cdot \text{Embed}$ ]
```

```
  L --> M
```

```
  M --> N[Top-K SearchHit<br/>chunk_id, text, score]
```

```
```
```

## ## Quick Start

### ### 1. Install dependencies

```
```bash
```

```
pip install fastapi uvicorn[standard] rank-bm25 sentence-transformers scikit-learn
```

```
```
```

### ### 2. Run the service

```
```bash
```

```
uvicorn api:app --host 0.0.0.0 --port 8000 --reload
```

```
```
```

### ### 3. Ingest documents

```
```bash
```

```
curl -X POST "http://localhost:8000/ingest" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "title": "Nia Overview",
```

```
    "content": "Nia is an indexing service.\n\nIt builds BM25 and semantic indexes over code and docs."
  }
}
```

4. Search

```
```bash
curl "http://localhost:8000/search?q=indexing&k=3"
```
```

```
**Response:**
```json
{
 "query": "indexing",
 "results": [
 {
 "chunk_id": "uuid",
 "text": "Nia is an indexing service.",
 "score": 0.92,
 "metadata": {"uri": "memory://doc", "title": "Nia Overview"}
 }
]
}
```
```

API Reference

| Endpoint | Method | Description | Parameters |
|-----------------------|--------|-----------------|---------------------------------------|
| <code>/health`</code> | GET | Liveness check | - |
| <code>/ingest`</code> | POST   | Ingest document | <code>{title, content, uri}`</code> |
| <code>/search`</code> | POST   | Hybrid search   | <code>{query, k=5, alpha=0.6}`</code> |
| <code>/search`</code> | GET    | Quick search    | <code>?q=query&k=5`</code> |

File Structure

```
...
nia/
├── ingestion_pipeline.py # Loaders + chunkers
├── core_index.py        # BM25 + embeddings + hybrid search
├── api.py               # FastAPI service
├── build_index.py       # Offline index builder (example)
└── README.md           # This file
```

...

Production Roadmap

1. **Persistence**: Postgres for metadata + pgvector/Qdrant for embeddings
2. **Ingestion workers**: Celery/Ray for scalable pipeline
3. **Auth**: API keys + tenant isolation
4. **Observability**: Structured logs + Prometheus metrics
5. **MCP server**: Model Context Protocol integration
6. **Symbol index**: Code-aware parsing + reference graph

Tech Stack

| Component | Library | Purpose |
|-----------|-----------------------|----------------------|
| Web | FastAPI | HTTP API |
| Lexical | rank-bm25 | BM25 search |
| Semantic | sentence-transformers | Embeddings |
| Chunking | Custom | Paragraph/fixed-size |

License

MIT - see `LICENSE` file.

Built with ❤️ for code and docs search. Next: persistent storage + MCP server. [1][2][3]

Citations:

[1] How to Build a Semantic Search Engine in Python

<https://www.deepset.ai/blog/how-to-build-a-semantic-search-engine-in-python>

[2] How to Implement Semantic Search in Python Step by Step

<https://www.pingcap.com/article/semantic-search-python-step-by-step/>

[3] Tutorial: Preprocessing Different File Types - Haystack

https://haystack.deepset.ai/tutorials/30_file_type_preprocessing_index_pipeline

—

```

<mxGraphModel dx="1422" dy="794" grid="1" gridSize="10" guides="1" tooltips="1"
connect="1" arrows="1" fold="1" page="1" pageScale="1" pageWidth="1169" pageHeight="827"
math="0" shadow="0">
  <root>
    <mxCell id="0"/>
    <mxCell id="1" parent="0"/>
    <mxCell id="sources" value="Sources"
style="swimlane;fontStyle=1;childLayout=stackLayout;horizontal=1;startSize=26;fillColor=#dae8
fc;strokeColor=#6c8ebf;horizontalStack=0;resizeParent=1;resizeParentMax=0;resizeLast=0;coll
apsible=1;marginBottom=0;" vertex="1" parent="1">
      <mxGeometry x="40" y="50" width="200" height="200" as="geometry"/>
    </mxCell>
    <mxCell id="g1" value="Git Repos"
style="rounded=0;whiteSpace=wrap;html=1;fillColor=#f5f5f5;fontColor=#333333;strokeColor=#6
66666;" vertex="1" parent="sources">
      <mxGeometry y="30" width="180" height="40" as="geometry"/>
    </mxCell>
    <mxCell id="d1" value="Docs/SDK&#xa;Pages"
style="rounded=0;whiteSpace=wrap;html=1;fillColor=#f5f5f5;fontColor=#333333;strokeColor=#6
66666;" vertex="1" parent="sources">
      <mxGeometry y="80" width="180" height="40" as="geometry"/>
    </mxCell>
    <mxCell id="p1" value="PDF Manuals"
style="rounded=0;whiteSpace=wrap;html=1;fillColor=#f5f5f5;fontColor=#333333;strokeColor=#6
66666;" vertex="1" parent="sources">
      <mxGeometry y="130" width="180" height="40" as="geometry"/>
    </mxCell>
    <mxCell id="ingestion" value="Ingestion Pipeline"
style="swimlane;fontStyle=1;childLayout=stackLayout;horizontal=1;startSize=26;fillColor=#fff2cc
;strokeColor=#d6b656;horizontalStack=0;resizeParent=1;resizeParentMax=0;resizeLast=0;colla
psible=1;marginBottom=0;" vertex="1" parent="1">
      <mxGeometry x="300" y="50" width="220" height="280" as="geometry"/>
    </mxCell>
    <mxCell id="pipe" value="IngestionPipeline"
style="ellipse;whiteSpace=wrap;html=1;aspect=fixed;fillColor=#ffe6cc;strokeColor=#d79b00;"
vertex="1" parent="ingestion">
      <mxGeometry y="120" width="200" height="60" as="geometry"/>
    </mxCell>
    <mxCell id="niaidx" value="Discovery Index Hybrid Engine"
style="swimlane;fontStyle=1;childLayout=stackLayout;horizontal=1;startSize=26;fillColor=#e1d5
e7;strokeColor=#9673a6;horizontalStack=0;resizeParent=1;resizeParentMax=0;resizeLast=0;co
llapsible=1;marginBottom=0;" vertex="1" parent="1">
      <mxGeometry x="580" y="50" width="220" height="280" as="geometry"/>
    </mxCell>

```

```

    <mxCell id="chunks" value="Chunks Store"
style="rounded=1;whiteSpace=wrap;html=1;fillColor=#f8cecc;strokeColor=#b85450;" vertex="1"
parent="niaidx">
    <mxGeometry y="40" width="200" height="50" as="geometry"/>
</mxCell>
    <mxCell id="bm25" value="BM25 Lexical"
style="rounded=1;whiteSpace=wrap;html=1;fillColor=#f8cecc;strokeColor=#b85450;" vertex="1"
parent="niaidx">
    <mxGeometry y="100" width="200" height="40" as="geometry"/>
</mxCell>
    <mxCell id="api" value="HTTP API FastAPI"
style="swimlane;fontStyle=1;childLayout=stackLayout;horizontal=1;startSize=26;fillColor=#d5e8
d4;strokeColor=#82b366;horizontalStack=0;resizeParent=1;resizeParentMax=0;resizeLast=0;co
llapsible=1;marginBottom=0;" vertex="1" parent="1">
    <mxGeometry x="860" y="50" width="200" height="180" as="geometry"/>
</mxCell>
    <mxCell id="ingest" value="POST /ingest"
style="ellipse;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d79b00;" vertex="1"
parent="api">
    <mxGeometry y="40" width="180" height="50" as="geometry"/>
</mxCell>
    <mxCell id="search" value="GET/POST /search"
style="ellipse;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d79b00;" vertex="1"
parent="api">
    <mxGeometry y="100" width="180" height="50" as="geometry"/>
</mxCell>
    <mxCell id="clients" value="Clients & Agents"
style="swimlane;fontStyle=1;childLayout=stackLayout;horizontal=1;startSize=26;fillColor=#fff2cc
;strokeColor=#d6b656;horizontalStack=0;resizeParent=1;resizeParentMax=0;resizeLast=0;collap
sible=1;marginBottom=0;" vertex="1" parent="1">
    <mxGeometry x="300" y="400" width="760" height="180" as="geometry"/>
</mxCell>
    <mxCell id="cursor" value="Cursor"
style="rounded=0;whiteSpace=wrap;html=1;fillColor=#e1f5fe;strokeColor=#0288d1;" vertex="1"
parent="clients">
    <mxGeometry y="30" width="140" height="40" as="geometry"/>
</mxCell>
    <mxCell id="g1-1" value=""
style="edgeStyle=orthogonalEdgeStyle;rounded=0;orthogonalLoop=1;jettySize=auto;html=1;str
okeColor=#666666;" edge="1" parent="1" source="g1" target="pipe">
    <mxGeometry relative="1" as="geometry"/>
</mxCell>

```

```

    <mxCell id="d1-2" value=""
style="edgeStyle=orthogonalEdgeStyle;rounded=0;orthogonalLoop=1;jettySize=auto;html=1;strokeColor=#666666;" edge="1" parent="1" source="d1" target="pipe">
    <mxGeometry relative="1" as="geometry"/>
</mxCell>
    <mxCell id="pipe-3" value=""
style="edgeStyle=orthogonalEdgeStyle;rounded=0;orthogonalLoop=1;jettySize=auto;html=1;entryX=0;entryY=0.5;entryDx=0;entryDy=0;strokeColor=#666666;" edge="1" parent="1"
source="pipe" target="chunks">
    <mxGeometry relative="1" as="geometry">
        <mxPoint x="580" as="sourcePoint"/>
    </mxGeometry>
</mxCell>
    <mxCell id="ingest-4" value=""
style="edgeStyle=orthogonalEdgeStyle;rounded=0;orthogonalLoop=1;jettySize=auto;html=1;exitX=1;exitY=0.5;exitDx=0;exitDy=0;strokeColor=#666666;" edge="1" parent="1" source="ingest"
target="pipe">
    <mxGeometry relative="1" as="geometry"/>
</mxCell>
    <mxCell id="search-5" value=""
style="edgeStyle=orthogonalEdgeStyle;rounded=0;orthogonalLoop=1;jettySize=auto;html=1;strokeColor=#666666;" edge="1" parent="1" source="search" target="bm25">
    <mxGeometry relative="1" as="geometry">
        <Array as="points">
            <mxPoint x="980" y="325"/>
            <mxPoint x="700" y="325"/>
        </Array>
    </mxGeometry>
</mxCell>
</root>
</mxGraphModel>

```

—

 Just built **Discovery Search** – a hybrid indexing & retrieval service that makes searching your codebases, docs, and technical content *blazing fast and precise*!

Next Step after Data Extraction using VLM based OCR .

Here's what makes it special:

****Core Features.****

 Ingests from strings, files (txt/md/pdf), directories

- ✓ Smart chunking: paragraph-aware + fixed-size sliding windows
- ✓ ****Hybrid search****: BM25 lexical + semantic embeddings (fusion scoring)
- ✓ Clean FastAPI API: `/ingest` → `/search` in seconds

****The Magic:****

...

Query: "how to implement indexing pipeline"

→ Returns exact chunks + metadata + relevance scores

...

****Architecture at a glance:****

...

Docs/Code → Ingestion Pipeline → Chunks → BM25 + Embeddings → Hybrid Ranker → Precise Results

...

****Why this matters for devs & teams:****

- Tired of grep + Google across scattered repos/docs?
- Need symbol-aware search but embeddings miss exact matches?
- Want one service for Cursor, Claude Code, **and** your custom agents?

****Next steps in roadmap:****

- Postgres/pgvector persistence
- MCP server for agent integration
- Code symbol extraction + reference graphs
- Distributed ingestion workers

#RAG #SemanticSearch #FastAPI #Python #AIEngineering #CodeIntelligence